

Go — The Complete Guide · Reference Notes

Curated from Maximilian Schwarzmüller — "Go: The Complete Guide" (Udemy, 12 sections / 199 lectures / ~15h 22m)

Code snapshots: github.com/mschwarzmueller/go-complete-guide-resources

Notes written for someone who already ships Go. Course content is covered in full, but the framing is

"what does this actually mean at runtime" rather than "here is what a variable is."

Compiled 2026-08-27 · verified against Go 1.24

How to read this

Every section is numbered **S01–S12** matching the course sections, so you can watch a lecture and jump straight to the matching notes. Inside each section:

MARKER	MEANING
	The mental model — what's actually happening in memory / at compile time
	Gotcha that bites people in production
	Course code does it this way; here's what you'd do in a real service
	Worth memorising verbatim

The course targets Go 1.21. Where Go has moved on (1.22 loop vars, `any`, `errors.Is/As`, `min/max`, range-over-func, `slices/maps` packages), it's flagged with .

Table of contents

- [S00 · Orientation: the Go model in one page](#)
 - [S01 · Getting Started](#)
 - [S02 · Go Essentials](#)
 - [S03 · Packages & Modules](#)
 - [S04 · Pointers](#)
 - [S05 · Structs & Custom Types](#)
 - [S06 · Interfaces & Generics](#)
 - [S07 · Arrays, Slices & Maps](#)
 - [S08 · Functions Deep Dive](#)
 - [S09 · Project: Price Calculator](#)
 - [S10 · Concurrency](#)
 - [S11 · Project: Event Booking REST API](#)
 - [S12 · Beyond the Course](#)
 - [Appendix A · Cheat sheet](#)
 - [Appendix B · The gotcha table](#)
 - [Appendix C · Standard library map](#)
-

S00 · Orientation: the Go model in one page

 **Go's whole design is one trade repeated everywhere: give up expressiveness, buy back readability and compile speed.** No inheritance, no exceptions, no generics until 1.18, no operator overloading, one loop keyword. Once you internalise that Go is *deliberately* under-featured, most "why can't I..." questions answer themselves.

Five facts that explain 90% of Go's behaviour:

1. **Everything is passed by value.** Always. Structs are copied, arrays are copied, slices are copied — but a slice *value* is a 3-word header pointing at a shared array, so copying it feels like a reference. Pointers exist to opt out of copying.
2. **Zero values are always valid and always defined.** `0`, `""`, `false`, `nil`. There is no uninitialised memory. `var x []int` is a usable nil slice; `var m map[string]int` is a readable but **not writable** nil map (see S07).
3. **Errors are values, not control flow.** A function that can fail returns `(T, error)`. `panic` is for "this program is broken", not "the user typed garbage."
4. **Capitalisation is the access modifier.** `Exported` vs `unexported`, at package granularity. There is no `private`/`protected`/`friend`.
5. **Interfaces are satisfied implicitly and defined by the consumer.** A type never declares "I implement X". The package that *needs* the behaviour declares the interface.

Composition over inheritance, concretely: Go gives you embedding (S05) for reuse of *implementation* and interfaces (S06) for reuse of *contract*. That's the whole object system.

S01 · Getting Started

9 lectures · ~14 min · repo: `code/01-getting-started/first-app`

1.1 What Go is, per the course slides

Google-built, open source. The pitch, four bullets:

- **Simplicity, clarity, scalability** — small spec, one obvious way to do things
- **High performance + concurrency-first** — compiled to native code, goroutines as a language primitive
- **Batteries included** — the stdlib genuinely covers HTTP servers, JSON, crypto, templating, testing
- **Static typing** — errors caught at compile time

Positioned as "syntax inspired by Python, semantics closer to C." Slides list the use cases:

networking & APIs, microservices, CLI tools.

 The real reason Go won in infra: a single statically-linked binary with no runtime dependency, fast compiles, and a GC that's tuned for low latency rather than throughput. Docker, Kubernetes, Terraform, Prometheus, etcd — all Go, all for those reasons.

1.2 Toolchain — the commands that matter

```

go mod init example.com/my-app    # creates go.mod; the
module path is the import prefix
go run .                          # compile + execute,
binary discarded
go run main.go                    # only works if main.go
needs no sibling files
go build                          # produces ./my-app in
cwd
go build -o bin/app ./cmd/api     # real-world form
go install                        # builds and drops binary
in $GOPATH/bin
go get github.com/some/pkg       # add a dependency
go mod tidy                       # add missing / drop
unused deps — run before every commit
go vet ./...                     # static analysis;
catches printf mistakes, lock copies
go fmt ./...                     # gofmt; non-negotiable,
no style debates in Go
go test ./...                    # runs *_test.go

```

 `go run main.go` failing with *"undefined: someFunc"* when the function is right there in

`helper.go` is the #1 beginner trap. `go run .` compiles the whole package; `go run main.go` compiles exactly that one file. **Always use `go run .`**

 Cross-compilation is one env var away, which is why Go dominates CI/CD tooling:

```

GOOS=linux GOARCH=arm64 go build -o app-linux-arm64 .
CGO_ENABLED=0 go build -ldflags="-s -w" -o app .    #
static, stripped, tiny — for scratch containers

```

1.3 The anatomy of every Go program

```

package main          // 1. package clause — mandatory,
first non-comment line

import "fmt"           // 2. imports — must all be used
or it won't compile

func main() {          // 3. entrypoint — only in package
main, no args, no return
    fmt.Println("Hello, Go!")
}

```

Four rules that are compiler errors, not warnings:

RULE	WHY GO DOES THIS
Unused import → error	Keeps builds fast and dependency graphs honest
Unused local variable → error	Almost always a bug or leftover debug code
Opening brace must be on the same line	Automatic semicolon insertion breaks otherwise
Formatting is <code>gofmt</code> 's job	Ends all style arguments permanently

⚠ The brace rule is not stylistic. Go's lexer inserts a semicolon at the end of any line ending in an identifier, literal, or `}/)`. So:

```

func main()
{           // becomes: func main(); { →
"unexpected semicolon"
}

```

Unused *package-level* variables and unused *struct fields* are fine — the rule is only for imports and function-local variables. Use `_` to deliberately discard:

```

_ = someValue // silence "declared and
not used"
import _ "github.com/lib/pq" // blank import: run init()
for its side effects only (see S11)

```

S02 · Go Essentials

44 lectures · ~3h 18m · repo: [code/02-go-essentials/01...25](#) · projects: *investment calculator, bank app, profit calculator*

The longest section in the course. Builds an interactive **Go Bank** CLI incrementally — variables → functions → control flow → file I/O → errors — which is a good spine because each concept lands in running code.

2.1 The four types you'll use 95% of the time

TYPE	HOLDS	ZERO VALUE	LITERALS
<code>int</code>	whole numbers, platform-width (64-bit on modern machines)	<code>0</code>	<code>-5</code> , <code>12</code> , <code>101</code>
<code>float64</code>	numbers with decimals, IEEE-754 double	<code>0.0</code>	<code>-15.9</code> , <code>0.0</code> , <code>23.12</code>
<code>string</code>	immutable UTF-8 bytes	<code>" "</code>	<code>"Hello"</code> , <code>`raw`</code>
<code>bool</code>	true/false	<code>false</code>	<code>true</code> , <code>false</code>

The full set you should know exists:

```

int8 int16 int32 int64 int          // int32 has alias
`rune` — a Unicode code point
uint8 uint16 uint32 uint64 uint     // uint8 has alias
`byte`
float32 float64
complex64 complex128               // yes, complex
numbers are built in. You will never use them.
string bool

```

📌 **Go has no implicit numeric conversion. None.** `int + float64` is a compile error.

```

var count int = 3
var price float64 = 10.5
// total := count * price          // ✗ invalid operation:
// mismatched types
total := float64(count) * price    // ✓ explicit
// conversion, always

```

⚠️ Converting a float **variable** to `int` **truncates** toward zero — it does not round. Use `math.Round` first if you want rounding.

```

f := 9.99
int(f)          // 9 — truncated
int(-9.99)      // ... but see below
int(math.Round(f)) // 10

```

⚠️ Subtle: `int(9.99)` with an **untyped constant** is a *compile error*, not a truncation —
cannot convert 9.99 (untyped float constant) to type int. Constant conversions must be exact;
only variable conversions truncate. Two different behaviours for identical-looking code.

⚠️ Integer division truncates: `5 / 2 == 2`. Write `5.0 / 2.0` or convert first.

Strings are immutable byte slices. `len(s)` returns *bytes*, not characters:

```
s := "héllö"
len(s)                // 6 – é is two bytes in UTF-8
len([]rune(s))        // 5 – actual character count
for i, r := range s { ... } // i jumps by byte width; r is a rune
s[0]                  // a byte (uint8), not a string
string(s[0])          // "h"
```

2.2 Declaring variables — four forms, one you'll use

```
var balance float64           // 1. zero
value: 0.0
var balance float64 = 1000.0   // 2. explicit
type + value
var balance = 1000.0           // 3. type
inferred → float64
balance := 1000.0              // 4. short
declaration — the idiomatic one
```

`:=` only works **inside a function**. At package level you must use `var`.

Grouped declaration keeps related config together:

```
var (
    accountBalance float64 = 1000
    accountOwner    string  = "Shadab"
    isActive        bool    // false
)
```

⚠️ **Inference defaults matter:** `x := 5` gives `int`, `x := 5.0` gives `float64`. If you need a `float64` seeded from a whole number, write `5.0` or `float64(5)` — a later `x / 2` on an `int` will truncate and the bug is silent.

⚠️ `:=` **requires at least one new variable on the left.** This is why the error-handling pattern works:

```
revenue, err := getUserInput("Revenue: ")    // both new
expenses, err := getUserInput("Expenses: ")  // ✓
expenses is new, err is reassigned
// err := something()                        // ✗ if err
already exists and it's the only var
```

2.3 Constants

```
const inflationRate = 2.5           // untyped constant —
adapts to context
const accountBalanceFile = "balance.txt"

const (
    StatusPending = "pending"
    StatusApproved = "approved"
)
```

🧠 **Untyped constants are more flexible than variables** — they have "arbitrary precision" until used:

```
const big = 1 << 62                // fine, no type yet
const ratio = 1 / 3.0              // computed exactly at compile
time
var f float64 = ratio               // takes on float64 here
var i int = 3                      // i is int
// var x = i * ratio               // ✗ ratio becomes float64,
mismatched with int
```

Constants can only hold values computable at compile time — no `time.Now()`, no slices, no maps.

iota for enums (not covered in the course, but you'll need it):

```
type Status int
const (
    Pending Status = iota    // 0
    Approved           // 1
    Rejected           // 2
)
func (s Status) String() string {
    return [...]string{"pending", "approved", "rejected"}[s]
}
```

2.4 Functions

```
func calculateFinancials(revenue, expenses, taxRate
float64) (float64, float64, float64) {
    ebt := revenue - expenses
    profit := ebt * (1 - taxRate/100)
    ratio := ebt / profit
    return ebt, profit, ratio
}
```

- Shared type on consecutive params: `(revenue, expenses, taxRate float64)`
- **Multiple return values** are the language's answer to exceptions and out-params
- Functions are package-scoped by default; capitalise to export

Named return values (course calls this "alternative return syntax"):

```
func calculateFinancials(revenue, expenses, taxRate
float64) (ebt, profit, ratio float64) {
    ebt = revenue - expenses
    profit = ebt * (1 - taxRate/100)
    ratio = ebt / profit
    return // "naked return" — returns the named vars
}
```



Use named returns for documentation, not for naked returns. `(int, error)` tells you nothing;

`(bytesWritten int, err error)` tells you everything. But naked `return` in a function longer than ~10

lines is genuinely hard to follow — write `return ebt, profit, ratio` explicitly. The one place naked

returns earn their keep is modifying a result inside `defer` (see S10.6).

2.5 Control flow

if — no parentheses, braces always required:

```
if depositAmount <= 0 {
    fmt.Println("Invalid amount.")
    continue
} else if depositAmount > limit {
    // ...
} else {
    // ...
}
```



if with initialiser — scopes a variable to the branch. This is *the* Go idiom:

```
if err := doThing(); err != nil {
    return err
}
// err does not exist here — can't accidentally check a
// stale error
```

for — the only loop keyword. Four shapes:

```
for i := 0; i < 10; i++ { }           // classic
for i < 10 { }                       // while
for { }                             // infinite – break to
exit
for i, v := range collection { }     // range (S07)
```

`break` exits the loop; `continue` skips to the next iteration. In the bank app, `continue` after an invalid amount is what keeps the menu alive instead of killing the program.

⚠ **`break` inside `switch` breaks the switch, not the enclosing loop.** Use a label:

```
outer:
for _, row := range grid {
    for _, cell := range row {
        if cell == target {
            break outer    // exits both
        }
    }
}
```

switch — no fallthrough by default, that's the big difference from C:

```
switch choice {
case 1:
    fmt.Println("Your balance is", accountBalance)
case 2, 3:
    // multiple values per
case
    // ...
default:
    fmt.Println("Goodbye!")
    return
}
```

Tagless switch replaces if/else-if chains cleanly:

```
switch {
case score >= 90: grade = "A"
case score >= 80: grade = "B"
default:         grade = "F"
}
```

`fallthrough` exists if you really want C semantics, but it's rare and usually a smell.

2.6 User input

```
var choice int
fmt.Print("Your choice: ")
fmt.Scan(&choice)           // note the & — Scan needs a
                             pointer to write into
```

⚠ **`fmt.Scan` stops at whitespace.** `"My Note Title"` gives you `"My"`. The course hits this in S05 and switches to `bufio`:

```
reader := bufio.NewReader(os.Stdin)
text, err := reader.ReadString('\n')
if err != nil {
    return ""
}
text = strings.TrimSuffix(text, "\n")
text = strings.TrimSuffix(text, "\r")    // 📌 the \r trim
                                         is for Windows CRLF — don't skip it
```

🔧 In real CLIs use `bufio.Scanner` — it handles line splitting and strips the newline for you:

```
sc := bufio.NewScanner(os.Stdin)
for sc.Scan() {
    line := sc.Text()    // already newline-free
}
```

2.7 String formatting — the `fmt` family

```
fmt.Println("Balance:", balance)           //
spaces between args, newline at end
fmt.Print("Enter: ")                       //
no newline — for prompts
fmt.Printf("%.1f\n", ebt)                   //
formatted to stdout
s := fmt.Sprintf("EBT: %.1f\nProfit: %.1f\n", a, b) //
formatted to string ← the useful one
err := fmt.Errorf("parsing %q: %w", name, err) //
formatted error with wrapping
```

 Verb table — learn these six:

VERB	USE
<code>%v</code>	default format for any value — the everyday verb
<code>%+v</code>	struct with field names — the debugging verb
<code>%#v</code>	Go syntax representation — the "paste it back into code" verb
<code>%T</code>	the type of the value
<code>%d %s</code> <code>%t</code>	int, string, bool
<code>%.2f</code>	float with 2 decimals; <code>%8.2f</code> right-aligns in 8 columns
<code>%q</code>	quoted string — makes empty strings and spaces visible
<code>%w</code>	wrap an error (only in <code>fmt.Errorf</code>) — preserves the chain for <code>errors.Is</code>

Multi-line strings use backticks — raw literals, no escape processing:

```
fmt.Printf(`Hi there!  
    Balance: %.2f  
    Owner:    %s  
`, balance, owner)
```

⚠ Raw strings keep leading tabs/spaces exactly as written, so indentation leaks into the output.

2.8 Files

```
// Write — whole file at once  
os.WriteFile("results.txt", []byte(results), 0644)  
  
// Read — whole file at once  
data, err := os.ReadFile("balance.txt")  
balanceText := string(data)
```

📌 0644 is a Unix permission literal: leading 0 = octal, 6=rw for owner, 4=r for group, 4=r for others. Use 0600 for anything with secrets.

os.WriteFile/os.ReadFile load the whole file into memory — fine for config, wrong for a 4 GB log.

For streaming, the course uses bufio.Scanner in S09:

```

file, err := os.Open(path)
if err != nil { return nil, err }
defer file.Close()

scanner := bufio.NewScanner(file)
var lines []string
for scanner.Scan() {
    lines = append(lines, scanner.Text())
}
if err := scanner.Err(); err != nil {    // 📌 always
    check — Scan() returning false is ambiguous
    return nil, err
}

```

⚠️ `scanner.Scan()` returns `false` both at clean EOF *and* on error. Without `scanner.Err()` you

silently truncate files on a read failure. Also: `bufio.Scanner` has a **64 KB default line limit** —

a longer line stops the scan. Use `scanner.Buffer()` to raise it.

2.9 Errors — the core idiom

Go's contract: a function that can fail returns an **error** as its last value.

```

func getUserInput(infoText string) (float64, error) {
    var userInput float64
    fmt.Print(infoText)
    fmt.Scan(&userInput)

    if userInput <= 0 {
        return 0, errors.New("value must be a positive
number")
    }
    return userInput, nil
}

```

Caller side, every single time:

```
revenue, err := getUserInput("Revenue: ")
if err != nil {
    fmt.Println(err)
    return // 📌 handle and return — don't fall
    through
}
```

📌 **error** is just an interface:

```
type error interface {
    Error() string
}
```

Nothing magic. Anything with an `Error() string` method is an error.

panic vs returning an error — the course shows both, commented side by side:

```
if err != nil {
    fmt.Println(err)
    return
    // panic(err)    // crashes the program, prints a stack
    trace
}
```

🧠 **Rule: panic only when the program cannot meaningfully continue.**

Bad user input → error. Missing

required config at startup → panic is defensible. A nil map you were supposed to initialise → panic

means you have a bug. Library code should essentially never panic across an API boundary.

🔧 **Modern error handling the course predates.** This is the part worth updating in your own code:

```

// 1. Wrap with %w to preserve the chain and add context
if err != nil {
    return fmt.Errorf("loading balance from %s: %w", path,
err)
}

// 2. Sentinel errors — compare with errors.Is, never ==
var ErrInsufficientFunds = errors.New("insufficient
funds")

if errors.Is(err, ErrInsufficientFunds) { ... }
if errors.Is(err, os.ErrNotExist)          { ... } // works
through any depth of wrapping

// 3. Typed errors — extract with errors.As
type ValidationError struct {
    Field  string
    Reason string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("field %s: %s", e.Field, e.Reason)
}

var ve *ValidationError
if errors.As(err, &ve) {
    log.Printf("bad field: %s", ve.Field)
}

```

📌 Error strings by convention are **lowercase, no trailing punctuation** — because they get wrapped:

"loading config: opening file: permission denied". The course's `errors.New("Failed to find file.")`

would read `"...: Failed to find file.: ..."`. Small thing, but `go vet`-adjacent linters flag it.

⚠️ `errors.New` in a loop or hot path allocates every call. Define sentinels as package-level vars.

2.10 Section project: profit calculator

The capstone for S02 — validate input, compute, persist. Full source:

```
package main

import (
    "errors"
    "fmt"
    "os"
)

func main() {
    revenue, err := getUserInput("Revenue: ")
    if err != nil {
        fmt.Println(err)
        return
    }
    expenses, err := getUserInput("Expenses: ")
    if err != nil {
        fmt.Println(err)
        return
    }
    taxRate, err := getUserInput("Tax Rate: ")
    if err != nil {
        fmt.Println(err)
        return
    }

    ebt, profit, ratio := calculateFinancials(revenue,
    expenses, taxRate)

    fmt.Printf("%.1f\n", ebt)
    fmt.Printf("%.1f\n", profit)
    fmt.Printf("%.3f\n", ratio)
    storeResults(ebt, profit, ratio)
}
```

```

func storeResults(ebt, profit, ratio float64) {
    results := fmt.Sprintf("EBT: %.1f\nProfit: %.1f\nRatio:
%.3f\n", ebt, profit, ratio)
    os.WriteFile("results.txt", []byte(results), 0644)
}

func calculateFinancials(revenue, expenses, taxRate
float64) (float64, float64, float64) {
    ebt := revenue - expenses
    profit := ebt * (1 - taxRate/100)
    ratio := ebt / profit
    return ebt, profit, ratio
}

func getUserInput(infoText string) (float64, error) {
    var userInput float64
    fmt.Print(infoText)
    fmt.Scan(&userInput)

    if userInput <= 0 {
        return 0, errors.New("value must be a positive
number")
    }
    return userInput, nil
}

```

⚠ Note the repeated `if err != nil { ... return }` blocks — this is idiomatic Go, and the verbosity is the point. There's no early-exit sugar. What you can do is push the repetition down:

```
func readAll(prompts ...string) ([]float64, error) {
    vals := make([]float64, 0, len(prompts))
    for _, p := range prompts {
        v, err := getUserInput(p)
        if err != nil {
            return nil, fmt.Errorf("reading %q: %w", p, err)
        }
        vals = append(vals, v)
    }
    return vals, nil
}
```

🔧 Real bug in the course code worth spotting: if `taxRate == 100`, then `profit == 0` and `ratio := ebt / profit` yields `+Inf` — no panic, because float division by zero is legal IEEE-754.

Integer division by zero *does* panic. Two different failure modes for the same-looking expression.

S03 · Packages & Modules

9 lectures · ~24 min · repo: `code/03-working-with-packages/01...04`

3.1 The three-level hierarchy

```
Module    ← go.mod, the unit of versioning and dependency
            (one per repo, usually)
    └─ Package ← one directory, the unit of compilation and
                visibility
        └─ File  ← organisational only; the compiler
                concatenates files in a package
```

 A package is a directory. All `.go` files in one directory must declare the same package name.

Files in the same package see each other's identifiers with no import and no qualifier — that's why

`bank.go` can call `presentOptions()` defined in `communication.go`.

3.2 Splitting across files (same package)

`bank.go` and `communication.go`, both `package main`:

```
// communication.go
package main

import "fmt"

func presentOptions() {
    fmt.Println("What do you want to do?")
    fmt.Println("1. Check balance")
    fmt.Println("2. Deposit money")
    fmt.Println("3. Withdraw money")
    fmt.Println("4. Exit")
}
```

```
// bank.go
package main

func main() {
    presentOptions() // no import, no prefix — same
package
}
```

 This is why `go run main.go` fails but `go run .` works: the first compiles one file, the second compiles the package.

3.3 Splitting across packages

Directory `fileops/` containing `fileops.go` with `package fileops`:

```
package fileops

import (
    "errors"
    "fmt"
    "os"
    "strconv"
)

func GetFloatFromFile(fileName string) (float64, error) {
    data, err := os.ReadFile(fileName)
    if err != nil {
        return 1000, errors.New("failed to find file")
    }

    valueText := string(data)
    value, err := strconv.ParseFloat(valueText, 64)
    if err != nil {
        return 1000, errors.New("failed to parse stored
value")
    }
    return value, nil
}

func WriteFloatToFile(value float64, fileName string) {
    valueText := fmt.Sprintf(value)
    os.WriteFile(fileName, []byte(valueText), 0644)
}
```

Import path = **module path** + **directory path**:

```
// go.mod says: module example.com/bank
import "example.com/bank/fileops"

value, err := fileops.GetFloatFromFile("balance.txt")
```

⚠ The import path is the *directory*, the qualifier is the *package name declared in the files*. They usually match, and you should keep them matching — when they don't, you need a named import:

```
import fops "example.com/bank/fileops"
```

3.4 Exporting — capitalisation is the whole system

```
func GetFloatFromFile() {} // exported — visible outside
the package
func parseValue() {} // unexported — package-
private

type User struct {
    Name string // exported field — marshals to JSON,
visible outside
    email string // unexported — invisible outside, and
JSON *silently drops it*
}
```

📌 Applies to **everything**: functions, types, struct fields, methods, constants, package-level vars.

⚠ The JSON consequence trips everyone: `encoding/json` uses reflection, and reflection can't read unexported fields. A struct with lowercase fields marshals to `{}`. No error, no warning. (See S05.7.)

🧠 **Visibility is per-package, not per-type.** Any code in package `user` can read `User.email`, including a completely unrelated file. Go has no `private` at type granularity — the package *is* the encapsulation boundary. This pushes you toward small, focused packages.

3.5 Third-party packages

```
go get github.com/Pallinder/go-randomdata
```

```
import (  
    "fmt"  
  
    "example.com/bank/fileops"  
    "github.com/Pallinder/go-randomdata"  
)  
  
fmt.Println("Reach us 24/7", randomdata.PhoneNumber())
```

📌 **Import grouping convention** (gofmt/goimports enforces the sorting, you enforce the grouping):

```
import (  
    "fmt"                // 1. stdlib  
    "os"  
  
    "example.com/bank/fileops" // 2. your module  
    "github.com/gin-gonic/gin" // 3. third party  
)
```

`go.mod` and `go.sum`:

```
module example.com/bank

go 1.21.2

require github.com/Pallinder/go-randomdata v1.2.0
```

- `go.mod` — module path, Go version, direct + indirect requirements
- `go.sum` — cryptographic checksums of every dependency; **commit it**, it's your supply-chain integrity
- `// indirect` marks a dep not imported directly by your code

```
go mod tidy          # sync go.mod with actual imports — run
                      before committing
go mod download      # populate module cache (CI step)
go mod vendor        # vendor deps into ./vendor for
                      hermetic builds
go mod why <pkg>     # "why is this dependency here?"
go list -m all       # full dependency tree
```

⚠ The course's `go.mod` has `github.com/Pallinder/go-randomdata v1.2.0` `// indirect` even though it's imported directly — an artefact of `go get` without a follow-up `go mod tidy`. Harmless, but it means the file is lying about the dependency graph.

3.6 Package layout in real projects

🔧 The course keeps everything at the module root. For anything beyond a demo:

```

myservice/
├─ go.mod
├─ cmd/
│   └─ api/
│       └─ main.go          # package main — thin: wire
deps, start server
├─ internal/                # 📌 magic dir: importable
ONLY by this module
│   └─ config/
│   └─ handlers/
│   └─ store/
│   └─ domain/
├─ pkg/                     # optional: genuinely
reusable by other modules
└─ migrations/

```

📌 **internal/** is enforced by the compiler. `example.com/other` cannot import `example.com/mine/internal/store`, ever. It's the only true access control Go has above package level, and it's how you stop your implementation details becoming someone's API.

Naming conventions:

- Package names: short, lowercase, single word, no underscores — `http`, `strconv`, `fileops`
- No plurals, no `util`/`common`/`helpers` (they become dumping grounds)
- **Don't stutter:** `user.User` reads fine, `user.UserService` doesn't. The package name is already context — `http.Server`, not `http.HTTPServer`.

⚠️ **Import cycles are a hard compile error.** Package A importing B while B imports A won't build.

The fix is usually to extract shared types into a third package, or to define the interface in the consumer (S06) so the dependency arrow flips.

`init()` functions run automatically before `main()`, after package-level vars are initialised:

```
func init() {  
    // runs once per package, before main  
}
```

Used for driver registration — this is what a blank import triggers:

```
import _ "github.com/mattn/go-sqlite3" // registers  
"sqlite3" with database/sql, exports nothing
```

🔧 Avoid `init()` in your own code. It runs invisibly, can't return errors, and makes test setup

miserable. Prefer explicit constructor calls from `main`.

S04 · Pointers

10 lectures · ~23 min · repo: `code/04-understanding-pointers/01...04`

4.1 The model

Per the course slides: **a pointer is a variable that stores an address instead of a value.**

Code	Computer Memory			
age := 32	<table><tr><td>0xc000018050</td><td>32</td></tr></table>		0xc000018050	32
0xc000018050	32			
agePointer := &age				
	address	value		

Two operators, and they're inverses:

OPERATOR	NAME	DOES
<code>&x</code>	address-of	value $\rightarrow$ pointer
<code>*p</code>	dereference	pointer $\rightarrow$ value

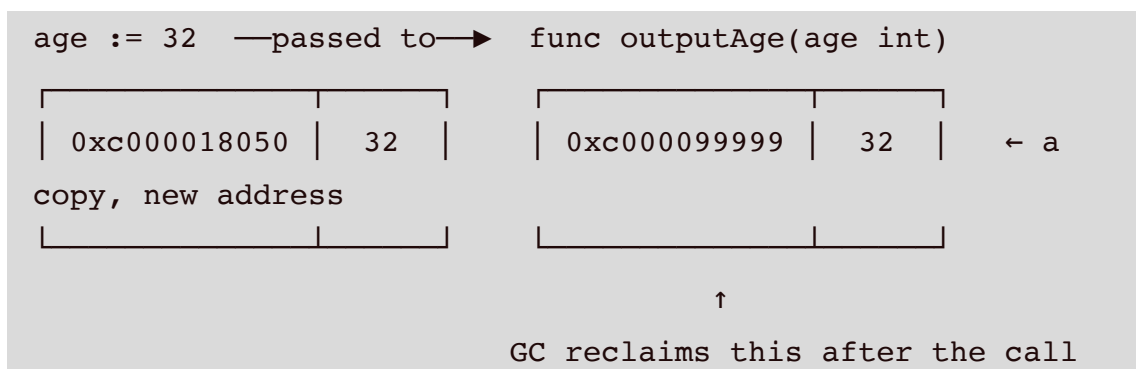
`*T` in a *type* position means "pointer to T". `*p` in an *expression* means "the value at p". Same symbol, two jobs.

```
age := 32           // int
var agePointer *int // declared, currently nil
agePointer = &age   // now points at age

fmt.Println(agePointer) // 0xc000018050 - the address
fmt.Println(*agePointer) // 32 - the value
*agePointer = 40        // writes through the pointer
fmt.Println(age)        // 40 - age itself changed
```

4.2 Why pointers exist – the course's two reasons

Reason 1: avoid copying. Go passes everything by value. Pass a 10 KB struct to a function and Go copies 10 KB.



Reason 2: mutate the caller's value.

```

func main() {
    age := 32
    editAgeToAdultYears(&age)
    fmt.Println(age)           // 14
}

func editAgeToAdultYears(age *int) {
    *age = *age - 18           // writes through — no return
    value needed
}

```

Without the pointer, the function mutates its own copy and the caller sees nothing.

 The slides' honest caveat: pointers *"can lead to less code, but also to less understandable code or unexpected behaviours."* That's the right instinct — pointers are a tool for two specific jobs, not a default.

4.3 nil, and the panic you'll actually hit

```

var p *int
fmt.Println(p)           // <nil>
fmt.Println(*p)          // 🌟 panic: runtime error: invalid
                           memory address or nil pointer dereference

```

The zero value of any pointer is `nil`. Dereferencing nil panics — this is Go's segfault, and it's the single most common runtime crash in Go services.

```

if p != nil {
    fmt.Println(*p)
}

```

⚠ Where nil pointers actually come from in production:

- A constructor that returns `(nil, err)` and the caller ignored `err`
- `json.Unmarshal` into a struct with pointer fields — absent JSON keys stay nil
- A map lookup of a missing key in a `map[string]*T` — returns nil, no panic until you deref
- A method on a nil receiver touching a field

⚠ **Nil interface vs nil pointer** — the classic Go trap, worth understanding once, properly:

```
type MyErr struct{}
func (e *MyErr) Error() string { return "boom" }

func doThing() error {
    var e *MyErr = nil
    return e           // 💣 returns a NON-nil error!
}

if err := doThing(); err != nil {
    // this branch RUNS
}
```

🧠 An interface value is a pair `(type, value)`. Returning a nil `*MyErr` gives `(*MyErr, nil)` — the type word is set, so the interface is not nil. **Fix: declare the return as the concrete type inside the function, or `return nil` explicitly.**

4.4 `new` and pointer creation

```

p := new(int)           // allocates a zeroed int, returns *int
*p = 42

u := &User{Name: "Shadab"} //  the idiomatic form
for structs — & on a composite literal

u2 := new(User)          // equivalent to &User{}

```

 You'll basically never write `new(). &T{}` is the way.

4.5 Pointers vs values — the decision table

SITUATION	USE
Small struct ($\leq \sim 3$ words), read-only	value
Large struct, or copied in a hot loop	pointer
Need to mutate the caller's data	pointer
Type has <i>any</i> pointer method	pointer everywhere — stay consistent
Contains a <code>sync.Mutex</code> / <code>sync.WaitGroup</code>	pointer always — copying a lock is a bug
Slices, maps, channels, funcs	value — already reference-like, <code>*[]T</code> is almost always wrong

 `*[]int` appears in the course (S08 `transformNumbers(numbers *[]int, ...)`) and needs the awkward `(*numbers)[0]`. That's a teaching device to reinforce pointers — **don't do it in real code**. A slice header is 3 words; passing it by value is cheap and the backing array is shared anyway. You only need `*[]T` if the function must `append` and have the caller see the new length.

 `go vet` catches copied mutexes: *"assignment copies lock value"*. Trust it, always.

🧠 **Escape analysis, briefly.** Go decides stack vs heap at compile time, not from `&`:

```
func f() *int {  
    x := 42  
    return &x    // perfectly safe — x "escapes", so it's  
                heap-allocated  
}
```

No dangling pointers, ever — this is legal Go and would be a use-after-free in C. Check what escapes:

```
go build -gcflags='-m' ./...
```

S05 · Structs & Custom Types

25 lectures · ~1h 49m · repo: `code/05-structs-custom-types/01...17` · project: *notes app*

5.1 Defining and instantiating

```
type User struct {  
    firstName string  
    lastName  string  
    birthDate string  
    createdAt time.Time  
}
```

Four ways to instantiate:

```

var u User // 1. zero
value — all fields zeroed
u := User{"Shadab", "K", "01/01/1999", time.Now()} // 2.
positional — ⚠ breaks on field reorder
u := User{ // 3. named
    fields — 📌 always use this
    firstName: "Shadab",
    lastName: "K",
}
u := &User{firstName: "Shadab"} // 4.
pointer to struct

```

⚠ Positional literals are a maintenance hazard: add a field in the middle and every call site silently shifts. `go vet` flags this for stdlib types (`composites` check) but not your own.

Struct comparison: structs are comparable with `==` if *all* fields are comparable. Slices, maps and funcs are not comparable, so a struct containing one isn't either.

```

type Point struct{ X, Y int }
Point{1, 2} == Point{1, 2} // true — field-by-field

type Bad struct{ Tags []string }
// Bad{} == Bad{} // ✗ compile error

```

5.2 Methods

A method is a function with a **receiver** — that's the entire difference.

```
func (u User) OutputUserDetails() {           // value
receiver — gets a COPY
    fmt.Println(u.firstName, u.lastName, u.birthDate)
}

func (u *User) ClearUserName() {             // pointer
receiver — can mutate
    u.firstName = ""
    u.lastName = ""
}
```

 **Value receiver mutations are invisible to the caller.** This is the #1 struct bug:

```
func (u User) ClearUserName() { // ← no *
    u.firstName = ""           // mutates the copy;
original untouched
}
```

 **Go auto-takes the address for you** when calling a pointer method on an *addressable* value:

```
u := User{...}
u.ClearUserName() // Go rewrites this as
(&u).ClearUserName()
```

This is why the distinction is easy to miss. But it does **not** happen for non-addressable values:

```
// users["a"].ClearUserName()    X map values aren't
addressable
// getUser().ClearUserName()    X function results
aren't addressable
```

 **The rule: pick one receiver kind per type and stick to it.** Mixed receivers create a subtle method-set problem (S06.3). Default to pointer receivers unless the type is small and immutable.

Methods on non-struct types — any type you *define* can have methods:

```
type str string

func (text str) log() {
    fmt.Println(text)
}

var name str = "Max"
name.log()
```

 You can only define methods on types declared **in your own package**.

`func (s string) foo()` is illegal — that's Go's answer to monkey-patching.

5.3 Constructors

Go has no constructors. The convention is a function named `New` (or `NewX`):

```

func New(firstName, lastName, birthdate string) (*User,
error) {
    if firstName == "" || lastName == "" || birthdate == ""
    {
        return nil, errors.New("first name, last name and
birthdate are required")
    }

    return &User{
        firstName: firstName,
        lastName:  lastName,
        birthDate: birthdate,
        createdAt: time.Now(),
    }, nil
}

```

Called as `user.New(...)` —  which is why the function is named `New` and not `NewUser`: the package name supplies the noun. `user.NewUser()` stutters.

Why a constructor at all: validation. Unexported fields + exported constructor = the only way to create a `User` is through code that validates it. That's Go's encapsulation idiom.

```

appUser, err := user.New(first, last, birthdate)
if err != nil {
    fmt.Println(err)
    return
}

```

 For structs with many optional fields, the functional-options pattern beats a 9-arg constructor:

```

type Option func(*Server)

```

```

func WithTimeout(d time.Duration) Option { return func(s
*Server) { s.timeout = d } }

func WithTLS(c *tls.Config) Option      { return func(s
*Server) { s.tls = c } }

func NewServer(addr string, opts ...Option) *Server {
    s := &Server{addr: addr, timeout: 30 * time.Second} //
defaults
    for _, opt := range opts {
        opt(s)
    }
    return s
}

srv := NewServer(":8080", WithTimeout(5*time.Second),
WithTLS(cfg))

```

5.4 Struct embedding

Go's composition mechanism. Declare a type **without a field name**:

```

type User struct {
    firstName string
    lastName  string
    birthDate string
    createdAt time.Time
}

type Admin struct {
    email    string
    password string
    User      // ← embedded, no field name
}

```

The embedded type's **fields and methods are promoted** to the outer type:

```

admin := user.NewAdmin("test@example.com", "test123")
admin.OutputUserDetails()    // promoted from User
admin.ClearUserName()        // promoted from User
admin.firstName              // promoted field access
admin.User.firstName         // the explicit path — always
                             available

```

Constructing an embedded struct uses the **type name as the field name**:

```

func NewAdmin(email, password string) Admin {
    return Admin{
        email:    email,
        password: password,
        User: User{                                // ← field key is the
type name
            firstName: "ADMIN",
            lastName:  "ADMIN",
            birthDate: "---",
            createdAt: time.Now(),
        },
    }
}

```

 **Embedding is not inheritance.** There's no polymorphism here: a `*User` variable can't hold an `Admin`, and `User`'s methods can't see `Admin`'s fields. It's syntactic sugar for delegation — the compiler writes `func (a Admin) OutputUserDetails() { a.User.OutputUserDetails() }` for you.

 **Method promotion respects receiver types.** `ClearUserName` has a `*User` receiver, so it's only in `*Admin`'s method set, not `Admin`'s. `admin.ClearUserName()` works because `admin` is addressable.

⚠️ **Ambiguous promotion** — embed two types with the same method name and the promotion is silently dropped. Calling it becomes a compile error until you disambiguate explicitly. Shallower depth wins over deeper.

🔧 The most common real use of embedding is **embedding an interface to get a partial implementation**:

```
type LoggingStore struct {
    Store          // embedded interface — LoggingStore
    satisfies Store automatically
    log *slog.Logger
}

func (l LoggingStore) Get(id string) (Item, error) {    //
    override just this one
    l.log.Info("get", "id", id)
    return l.Store.Get(id)
}
```

5.5 Struct tags

Backtick metadata on fields, read at runtime via reflection:

```
type Note struct {
    Title      string    `json:"title"`
    Content    string    `json:"content"`
    CreatedAt  time.Time `json:"created_at"`
}
```

📌 The **exact** syntax matters — reflection parses it as a string, so mistakes are silent:

```
Title string `json:"title"`           // ✓
Title string `json: "title"`         // ✗ space after colon –
silently ignored, field marshals as "Title"
Title string "json:\"title\""        // ✓ but nobody writes
this
```

Common tag options:

```
type User struct {
    ID          int64  `json:"id"`
    Name        string `json:"name,omitempty"` // omit if
zero value
    Password string `json:"- "`           // 📌 NEVER
marshal this field
    Internal string `json:"-,"`       // literally
the key "-" (obscure, avoid)
    Age         int    `json:"age,string"` // encode
number as JSON string
}
```

Multiple libraries, space-separated:

```
Email string `json:"email" db:"email_address"
validate:"required,email" binding:"required"`
```

⚠️ `json:"- "` on a password field is a **security control**, not a formatting choice. The course's REST API (S11) returns the `User` struct in responses; without `json:"- "` you leak the bcrypt hash.

5.6 The notes app — S05's project

```
package note

import (
```

```

"encoding/json"
"errors"
"fmt"
"os"
"strings"
"time"
)

type Note struct {
    Title      string    `json:"title"`
    Content    string    `json:"content"`
    CreatedAt  time.Time `json:"created_at"`
}

func (note Note) Display() {
    fmt.Printf("Your note titled %v has the following\ncontent:\n\n%v\n\n", note.Title, note.Content)
}

func (note Note) Save() error {
    fileName := strings.ReplaceAll(note.Title, " ", "_")
    fileName = strings.ToLower(fileName) + ".json"

    json, err := json.Marshal(note)
    if err != nil {
        return err
    }
    return os.WriteFile(fileName, json, 0644)
}

func New(title, content string) (Note, error) {
    if title == "" || content == "" {
        return Note{}, errors.New("invalid input")
    }
    return Note{Title: title, Content: content, CreatedAt: time.Now()}, nil
}

```

⚠ Two things in `Save()` worth flagging:

1. `json, err := json.Marshal(note)` — **the variable shadows the package name**. Legal, compiles, and you can never call `json.` anything again in that scope. Rename to `data`.
2. The filename comes from user input with only space-replacement. `../../etc/passwd` as a title is a **path traversal**. Real fix: `filepath.Base()` + an allowlist regex.

5.7 JSON marshalling rules

```
data, err := json.Marshal(note)           // struct → []byte
err = json.Unmarshal(data, &note)         // []byte → struct (note the &)

enc := json.NewEncoder(file)               // streaming — better for files/HTTP
err = enc.Encode(data)
dec := json.NewDecoder(r.Body)
err = dec.Decode(&target)
```

📌 The rules, in order of how often they bite:

1. **Only exported fields marshal**. Lowercase → silently absent.
2. **No tag** → the Go field name is the key, capital and all: `{"Title": ...}`.
3. **Unmarshal is case-insensitive** on keys; marshal is not.
4. **Unknown JSON keys are ignored** by default (`DisallowUnknownFields()` to error instead).
5. **Missing keys leave the zero value** — you cannot distinguish "absent" from "sent as 0/empty".
Use `*int` / `*string` when that distinction matters.

6. `time.Time` marshals as RFC 3339: `"2025-01-01T15:30:00Z"`.

🔧 Custom marshalling when the defaults don't fit:

```
func (n Note) MarshalJSON() ([]byte, error) {
    type Alias Note      // 📌 alias breaks the recursion —
    Alias has no MarshalJSON method
    return json.Marshal(struct {
        Alias
        Slug string `json:"slug"`
    }{Alias(n), slugify(n.Title)})
}
```

S06 · Interfaces & Generics

11 lectures · ~44 min · repo: `code/06-interfaces/01...09`

6.1 Interfaces are contracts, satisfied implicitly

```
type saver interface {
    Save() error
}
```

Any type with a `Save() error` method satisfies `saver`. **No implements keyword, no registration.**

`note.Note` and `todo.TODO` both satisfy it without knowing `saver` exists — and crucially, without importing the package that defines it.

```
func saveData(data saver) error {
    err := data.Save()
    if err != nil {
        fmt.Println("Saving failed.")
        return err
    }
    fmt.Println("Saving succeeded!")
    return nil
}

saveData(userNote)    // Note satisfies saver
saveData(todo)        // Todo satisfies saver
```

 **This inverts the dependency arrow, and it's the single most important idea in Go design.** In Java,

`Note implements Saver` means `note` must import `saver`. In Go, the *consumer* declares what it needs.

Your `store` package defines `type Cache interface { Get(string)`

`([]byte, bool) }` — Redis, memcached

and a test double all satisfy it, and none of them ever heard of your package.

 **"Accept interfaces, return structs."** Take the narrowest interface you need as a parameter; return concrete types so callers keep all the methods.

 **Define interfaces where they're used, not where they're implemented.**

A `models` package exporting

`UserRepository` is a Java habit. The `handlers` package that needs it should declare it.

 **Keep them tiny.** The stdlib's most-used interfaces have one method:

`error`, `io.Reader`,

`io.Writer`, `fmt.Stringer`, `sort.Interface` (three, and it shows). *"The bigger the interface, the*

weaker the abstraction." — Rob Pike.

6.2 Interface embedding

```
type saver interface {
    Save() error
}

type outputtable interface {
    saver          // embedded — pulls in Save() error
    Display()
}

// identical to:
// type outputtable interface {
//     Save() error
//     Display()
// }
```

Same mechanic as `io.ReadWriter`:

```
type ReadWriter interface {
    Reader
    Writer
}
```

Used to build a two-tier API — the wide interface for the caller, the narrow one for the helper:

```
func outputData(data outputtable) error {
    data.Display()
    return saveData(data)    // outputtable ⊇ saver, so
    this is fine
}
```

6.3 Method sets — the rule people get wrong

 A `*T` method is NOT in `T`'s method set.

```

type Counter struct{ n int }
func (c *Counter) Inc() { c.n++ }

type Incrementer interface{ Inc() }

var i Incrementer = Counter{}    // ✗ compile error:
method has pointer receiver
var i Incrementer = &Counter{}  // ✓

```

RECEIVER KIND	T SATISFIES?	*T SATISFIES?
func (t T) M()	✓	✓
func (t *T) M()	✗	✓

🧠 **Why:** to call `Inc()` on a `Counter` value stored inside an interface, Go would need its address — but interface storage isn't addressable. The direction that works always works; the other never does.

⚠️ This is why "pick one receiver kind and stay consistent" matters. Mix them and half your methods promote and half don't.

Compile-time assertion — put this next to your type so violations fail at build, not at runtime:

```

var _ Incrementer = (*Counter)(nil)    // 📌 costs nothing,
catches the error immediately

```

6.4 `any` / `interface{}` — the empty interface

```
func printSomething(value interface{}) { // 🛠️ Go 1.18+:
write `any` — exact alias, better name
    // ...
}

printSomething(1)
printSomething(1.5)
printSomething("Hello")
```

Zero methods → every type satisfies it → it's Go's `Object`.

⚠️ **`any` is not a type, it's a loss of type information.** You need a type assertion to get anything back out, and that check happens at runtime. Since generics landed, most uses of `any` are a mistake. The legitimate ones: `json.Unmarshal` targets, `fmt` verbs, `map[string]any` for genuinely dynamic data.

6.5 Type assertions and the comma-ok form

```
intVal, ok := value.(int)
if ok {
    fmt.Println("Integer:", intVal)
    return
}

floatVal, ok := value.(float64)
if ok {
    fmt.Println("Float:", floatVal)
    return
}
```

📌 Two forms, and the difference is a crash:

```
v := value.(int)           // panics if value isn't an int
v, ok := value.(int)       // ok=false, v=zero value. Never
                             panics. Use this.
```

Assertions also work interface → interface:

```
if s, ok := w.(io.Seeker); ok {    // does this Writer also
    seek?
    s.Seek(0, io.SeekStart)
}
```

🔧 That's how you do optional capabilities in Go — `http.ResponseWriter` and `http.Flusher` work exactly this way.

6.6 Type switches

The clean form of the assertion chain above:

```
switch v := value.(type) {          // 📌 the `.(type)`
    syntax is legal ONLY in a switch header
case int:
    fmt.Println("Integer:", v)      // v is typed int here
case float64:
    fmt.Println("Float:", v)        // v is float64 here
case string:
    fmt.Println("String:", v)
case nil:
    fmt.Println("nil interface")
case error:
    fmt.Println("an error:", v)     // interfaces work as cases
    too
default:
    fmt.Printf("unhandled type %T\n", v)
}
```

`v` is re-typed per case — that's the whole point over `if/else` chains.

⚠ In a multi-type case (`case int, int64:`) `v` stays the **interface type**, since there's no single type it could be.

6.7 Generics

The course introduces them at the end of S06:

```
func add[T int | float64 | string](a, b T) T {  
    return a + b  
}  
  
add(1, 2)           // T inferred as int → 3  
add(1.5, 2.5)       // float64 → 4.0  
add("go", "lang")   // string → "golang"
```

Anatomy: `[T int | float64 | string]` is the **type parameter list**; the union is the **constraint**.

📌 **Interfaces vs generics — the actual distinction:**

	INTERFACE	GENERIC
Resolved	runtime (dynamic dispatch)	compile time
Type known inside	no — need an assertion	yes, fully
Return type	fixed	can be the parameter type
Best for	<i>behaviour</i> varies	<i>type</i> varies, logic identical

`add` can't be written with interfaces: you'd take `any`, and then `a + b` won't compile because `any` has no `+`. And you couldn't return `T`.

Named constraints and the `constraints` idiom:

```

type Number interface {
    ~int | ~int64 | ~float64    // 📌 ~ means "any type
    whose UNDERLYING type is this"
}

func Sum[T Number](nums []T) T {
    var total T                // zero value of T
    for _, n := range nums {
        total += n
    }
    return total
}

type Celsius float64
Sum([]Celsius{1.5, 2.5})    // ✓ works because of ~float64;
without ~ it wouldn't

```

Built-in constraints:

```

any                // no constraint
comparable         // supports == and != —
required for map keys
cmp.Ordered        // <, <=, >, >= (Go 1.21; was
golang.org/x/exp/constraints.Ordered)

```

Generic types, not just functions:

```

type Stack[T any] struct{ items []T }

func (s *Stack[T]) Push(v T) { s.items = append(s.items,
v) }

func (s *Stack[T]) Pop() (T, bool) {
    var zero T
    if len(s.items) == 0 {
        return zero, false
    }
}

```

```

    v := s.items[len(s.items)-1]
    s.items = s.items[:len(s.items)-1]
    return v, true
}

s := &Stack[int]{}

```

 `var zero T` is the generic idiom for "the zero value of whatever T is" — you can't write `nil` or `0`.

 **Generic methods don't exist.** A method can't add its own type parameter beyond the receiver's.

```

// func (s *Stack[T]) Map[U any](f func(T) U) *Stack[U]
* not legal Go
func Map[T, U any](s *Stack[T], f func(T) U) *Stack[U]
// ✓ free function

```

 **When to reach for generics:** container types, and functions where the logic is genuinely type-independent. The stdlib now ships the useful ones — `slices.Sort`, `slices.Contains`, `slices.Index`, `maps.Keys`, `maps.Values`, `cmp.Compare`, `min/max` builtins (1.21). Check there before writing your own.

Rob Pike's rule, still correct: *"Write code, don't design types."* Write it concrete first. If you

write the same function a third time for a third type, then generify.

S07 · Arrays, Slices & Maps

17 lectures · ~1h 43m · repo: [code/07-arrays-slices-maps/01...15](#)

The section with the most runtime surprises. Understand the slice header and most of them evaporate.

7.1 Arrays — fixed length, and length is part of the type

```
var productNames [4]string = [4]string{"A Book"}    //
remaining 3 are ""
prices := [4]float64{10.99, 9.99, 45.99, 20.0}
prices[2] = 45.99
fmt.Println(len(prices))    // 4 — always, it's in the type
```

 `[4]int` and `[5]int` are different types. A function taking `[4]int` cannot accept `[5]int`.

This is why arrays are almost never in Go APIs.

Arrays are **values** — assigning or passing copies every element:

```
a := [3]int{1, 2, 3}
b := a    // full copy
b[0] = 99
fmt.Println(a)    // [1 2 3] — untouched
```

Let the compiler count:

```
nums := [...]int{1, 2, 3, 4}    // [4]int
```

 You'll use arrays for: fixed-size crypto output (`[32]byte` from SHA-256), lookup tables, and as map keys (arrays are comparable, slices aren't). Everything else is a slice.

7.2 Slices — the 3-word header

 A slice is this struct, and everything follows from it:

```
type slice struct {  
    ptr *T    // pointer to the first element in the backing  
    array  
    len int    // elements currently visible  
    cap int    // elements from ptr to the end of the backing  
    array  
}
```

Slicing shares the backing array — it does not copy. The course demo, annotated:

```
prices := [4]float64{10.99, 9.99, 45.99, 20.0}  
  
featuredPrices := prices[1:]           // len 3, cap 3  
→ [9.99 45.99 20]  
featuredPrices[0] = 199.99             // ⚠ writes  
into `prices`  
// prices is now [10.99 199.99 45.99 20]  
  
highlightedPrices := featuredPrices[:1] // len 1, cap 3  
→ [199.99]  
len(highlightedPrices), cap(highlightedPrices) // 1, 3  
  
highlightedPrices = highlightedPrices[:3] // 📌 RE-EXTEND  
— legal up to cap  
// → [199.99 45.99 20], len 3, cap 3
```

 **cap** is measured from the slice's start to the end of the backing array, which is why

`highlightedPrices` has cap 3 despite len 1 — and why you can re-grow it with `[:3]`. Data "past the end" is still reachable. That's a feature and a footgun.

Slice expressions:

```
s[low:high]          // len = high-low,  cap = cap(s)-low
s[low:]              // to the end
s[:high]             // from the start
s[:]                 // whole thing — the array → slice
conversion
s[low:high:max]      // 📌 3-index: caps the capacity at
max-low
```

🔧 The 3-index form is how you hand out a slice safely:

```
func (b *Buf) Bytes() []byte {
    return b.data[0:b.n:b.n]    // cap == len, so caller's
    append MUST reallocate
}
```

7.3 Literals, append, and the reallocation rule

```
prices := []float64{10.99, 8.99}          // no size →
slice, not array
prices[1] = 9.99
prices = append(prices, 5.99, 12.99, 29.99, 100.10)
prices = prices[1:]                       // drop the
first element
```

📌 **append** returns a new slice header — you must reassign. `append(s, x)` without assignment is almost always a bug (and `go vet` catches it).

🧠 What **append** actually does:

```

if len < cap: write in place at index len, return header
with len+1
                ⚠ this MUTATES the shared backing array
else:          allocate a bigger array (~2× under 256
elems, ~1.25× above),
                copy everything, return a header pointing
at the NEW array
                ⚠ the slice is now DISCONNECTED from any
other slice of the old array

```

⚠ **The aliasing bug this creates** — the one that gets people in code review:

```

a := []int{1, 2, 3, 4, 5}
b := a[1:3]                // len 2, cap 4
b = append(b, 99)          // fits in cap → writes a[3]!
fmt.Println(a)             // [1 2 3 99 5] ← a was
modified
b = append(b, 100)         // still fits (cap 4, len 3) →
writes a[4]
fmt.Println(a)             // [1 2 3 99 100]
b = append(b, 101)         // exceeds cap → reallocates,
b detaches
b[0] = -1
fmt.Println(a)             // [1 2 3 99 100] — no longer
affected

```

📌 **Whether append mutates the parent depends on capacity, which depends on history.** Never rely on either behaviour. To guarantee a detached copy:

```

dst := make([]int, len(src))
copy(dst, src)
// or, Go 1.21+:
dst := slices.Clone(src)

```

Spreading:

```
discountPrices := []float64{101.99, 80.99, 20.59}
prices = append(prices, discountPrices...) // ...
unpacks the slice
```

`append([]byte, string...)` is a special case that works.

7.4 `make` — preallocation

```
userNames := make([]string, 2, 5) // len 2, cap 5 → ["",
"" ]
userNames[0] = "Julie"
userNames = append(userNames, "Max") // → ["Julie",
"", "Max"], len 3
userNames = append(userNames, "Manuel") // → len 4
```

 **`make([]T, n)` gives you `n` zero-valued elements, not an empty slice with room for `n`.** The classic bug:

```
s := make([]int, 5)
s = append(s, 1) // → [0 0 0 0 0 1] — probably not
what you wanted
s := make([]int, 0, 5)
s = append(s, 1) // → [1] ✓
```

 **Preallocate whenever you know the size.** This is the single highest-leverage Go optimisation:

```
out := make([]Result, 0, len(input)) // one allocation
instead of log2(n) + copies
for _, in := range input {
    out = append(out, transform(in))
}
```

nil slice vs empty slice:

```
var a []int           // nil:  a == nil, len 0, no
                        allocation
b := []int{}          // empty: b != nil, len 0, allocated
                        header
c := make([]int, 0)    // empty
```

Both are usable — `len`, `range` and `append` all work on nil slices.  **Prefer `var a []int`.**

The one place it matters is JSON: nil marshals to `null`, empty marshals to `[]`. Frontends care.

7.5 Maps

```
websites := map[string]string{
    "Google":           "https://google.com",
    "Amazon Web Services": "https://aws.com",
}

fmt.Println(websites["Amazon Web Services"])
websites["LinkedIn"] = "https://linkedin.com" // insert
or update
delete(websites, "Google")                   // delete
— no return value
len(websites)

courseRatings := make(map[string]float64, 3) // 3 =
capacity HINT, not a limit
```

 **The comma-ok idiom is mandatory.** A missing key returns the *zero value*, indistinguishable from a real zero:

```

rating := courseRatings["rust"]           // 0.0 — but
does the key exist?
rating, ok := courseRatings["rust"]       // ok ==
false ← the answer
if _, exists := courseRatings["go"]; exists { ... }

```

⚠ **Writing to a nil map panics.** Reading from one does not.

```

var m map[string]int
fmt.Println(m["x"])    // 0 — fine
fmt.Println(len(m))    // 0 — fine
m["x"] = 1             // 💣 panic: assignment to entry in
nil map

```

This bites via struct fields: a struct with a `map` field has a nil map after `var s MyStruct`.

Initialise in your constructor.

⚠ **Map iteration order is deliberately randomised.** Not "unspecified" — actively shuffled per run so you can't accidentally depend on it. For deterministic output, sort the keys:

```

keys := make([]string, 0, len(m))
for k := range m {
    keys = append(keys, k)
}
sort.Strings(keys)           // or slices.Sort(keys)
for _, k := range keys {
    fmt.Println(k, m[k])
}

```

⚠ **Map values are not addressable.**

```

type P struct{ X int }
m := map[string]P{"a": {1}}
// m["a"].X = 5           X compile error
// (&m["a"]).X = 5       X cannot take address

p := m["a"]; p.X = 5; m["a"] = p    // ✓ read-modify-write
m2 := map[string]*P{"a": {1}}; m2["a"].X = 5    // ✓ map of
pointers

```

⚠ **Maps are NOT goroutine-safe.** Concurrent read+write triggers `fatal`

`error: concurrent map writes`

— which is a *runtime crash you cannot recover from*, not a panic. Use

`sync.RWMutex` OR `sync.Map`

(only for the append-only / disjoint-key patterns it's tuned for).

📌 **Key types must be comparable:** no slices, maps, or funcs as keys. Structs and arrays of comparable types are fine.

7.6 Maps vs structs

	STRUCT	MAP
Keys	fixed at compile time	dynamic
Types	per-field	one value type for all
Typos	compile error	silent zero value
Method-able	yes	yes, via a named type
Use for	known shape: a <code>User</code> , a config	dynamic sets: counters, indexes, caches

📌 If you know the fields, use a struct. `map[string]any` for structured data means giving up the type system for nothing.

7.7 Named types over collections

```
type floatMap map[string]float64

func (m floatMap) output() {
    fmt.Println(m)
}

courseRatings := make(floatMap, 3)
courseRatings["go"] = 4.7
courseRatings.output()
```

 Course calls these "type aliases" but they're **type definitions** — a genuinely new type. Real aliases use `=` and are a different feature:

```
type MyMap map[string]int    // definition: new type,
                             // methods allowed, needs conversion
type MyMap = map[string]int  // alias: literally the same
                             // type, for gradual refactors
```

Same trick gives you `sort.Interface`:

```
type ByAge []Person
func (a ByAge) Len() int           { return len(a) }
func (a ByAge) Swap(i, j int)      { a[i], a[j] = a[j], a[i] }
func (a ByAge) Less(i, j int) bool { return a[i].Age < a[j].Age }

sort.Sort(ByAge(people))
//  Go 1.21+ makes this obsolete:
slices.SortFunc(people, func(a, b Person) int { return
cmp.Compare(a.Age, b.Age) })
```

7.8 range

```
for index, value := range userNames { ... } //  
slice/array: index, element  
for key, value := range courseRatings { ... } // map: key,  
value (RANDOM ORDER)  
for i, r := range "hélllo" { ... }           // string:  
byte offset, rune  
for v := range ch { ... }                   // channel:  
values until closed (S10)  
for i := range 10 { ... }                   // 🔧 Go  
1.22+: integer range, 0..9
```

Drop what you don't need:

```
for _, value := range items { } // value only  
for index := range items { }   // index only  
for range items { }           // just count iterations
```

⚠️ **range** copies each element. Mutating `value` does nothing:

```
for _, u := range users {  
    u.Name = "x" // modifies the copy — users  
    unchanged  
}  
for i := range users {  
    users[i].Name = "x" // ✓ index into the original  
}
```

This also means ranging over a slice of large structs copies each one — use the index for big elements.

⚠️ **range** evaluates the slice expression once, so appending inside the loop doesn't extend it:

```
s := []int{1, 2, 3}
for _, v := range s {
    s = append(s, v)    // terminates after 3 iterations
}
```

 **The loop variable capture bug — fixed in Go 1.22, but know it:**

```
// Go < 1.22: ONE variable `v` reused across iterations
for _, v := range items {
    go func() { fmt.Println(v) }()    // all goroutines
    likely print the LAST item
}
// Old fix:
for _, v := range items {
    v := v                            // shadow into a per-
    iteration variable
    go func() { fmt.Println(v) }()
}
```

 **Go 1.22+ gives each iteration a fresh variable**, so the naive version is now correct — but only if `go.mod` declares `go 1.22` or later. The language version is per-module, so an old `go.mod` keeps old

semantics even on a new toolchain. Check yours.

S08 · Functions Deep Dive

8 lectures · ~57 min · repo: `code/08-functions-deep-dive/01...06`

8.1 Functions are values

```
func double(number int) int { return number * 2 }
func triple(number int) int { return number * 3 }

var f func(int) int = double
f(5)    // 10
```

Passed as arguments:

```
func transformNumbers(numbers []int, transform func(int)
int) []int {
    dNumbers := []int{}
    for _, val := range numbers {
        dNumbers = append(dNumbers, transform(val))
    }
    return dNumbers
}

doubled := transformNumbers(&numbers, double)
tripled := transformNumbers(&numbers, triple)
```

Named function types clean up the signature — worth it as soon as the type appears twice:

```
type transformFn func(int) int

func transformNumbers(numbers []int, transform
transformFn) []int { ... }
```

The course's own comment shows why:

```
type anotherFn func(int, []string, map[string][]int)
([]int, string)
```

Returned from functions:

```
func getTransformerFunction(numbers []*int) transformFn {
    if (*numbers)[0] == 1 {
        return double
    }
    return triple
}
```

🔧 Again: `*[]int` here is pedagogical. Real signature is `numbers []int` — and `(*numbers)[0]` becomes `numbers[0]`, which is the point.

📌 Function types are the reason Go doesn't need most of the strategy/command/callback patterns.

`http.HandlerFunc` is the canonical example — a *function type* with a method, so a bare function satisfies the `http.Handler` interface:

```
type HandlerFunc func(ResponseWriter, *Request)
func (f HandlerFunc) ServeHTTP(w ResponseWriter, r
*Request) { f(w, r) }
```

8.2 Anonymous functions

Declared inline, no name:

```
transformed := transformNumbers(&numbers, func(number int)
int {
    return number * 2
})
```

Immediately invoked (IIFE):

```
result := func(a, b int) int { return a + b }(3, 4) // 7
```

Assigned to a variable:

```
greet := func(name string) { fmt.Println("Hi", name) }  
greet("Shadab")
```

⚠️ An anonymous function assigned to a variable **can't recurse by name** — the variable isn't in scope inside its own literal. Declare it first:

```
var fib func(int) int  
fib = func(n int) int {  
    if n < 2 { return n }  
    return fib(n-1) + fib(n-2)  
}
```

8.3 Closures — the important one

📌 A closure captures variables by reference, not by value. The inner function keeps the outer variable alive for as long as it exists.

```
func createTransformer(factor int) func(int) int {  
    return func(number int) int {  
        return number * factor    // `factor` outlives  
createTransformer  
    }  
}  
  
double := createTransformer(2)  
triple := createTransformer(3)  
  
transformNumbers(&numbers, double)    // [2 4 6]  
transformNumbers(&numbers, triple)    // [3 6 9]
```

🧠 Each call to `createTransformer` creates a fresh binding. `double` and `triple` hold separate `factor` variables — `factor` escapes to the heap, and the GC frees it when the closure is collected.

That's why `go build -gcflags='-m'` reports "moved to heap: factor".

Mutable captured state — closures over a variable share it:

```
func counter() (func() int, func()) {
    count := 0
    inc := func() int { count++; return count }
    reset := func() { count = 0 }
    return inc, reset          // both close over the SAME
count
}
```

⚠ **Capture-by-reference is exactly what makes the loop-variable bug bite** (S07.8). The closure holds the *variable*, so if the loop reuses one variable, every closure sees the final value.

🔧 Closures in real Go, the three patterns you'll actually write:

```
// 1. Middleware — the dominant HTTP pattern
func Logging(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        log.Printf("%s %s %v", r.Method, r.URL.Path,
time.Since(start))
    })
}

// 2. Deferred cleanup that reads a final value
func process() (err error) {
    defer func() {
        if err != nil { metrics.Failures.Inc() } // sees the
named return
    }()
    // ...
}
```

```
}
```

```
// 3. Functional options (S05.3)
```

8.4 Recursion

```
func factorial(number int) int {  
    if number == 0 {           // 📌 base case first, always  
        return 1  
    }  
    return number * factorial(number-1)  
}
```

Call stack, from the slides:

```
factorial(5) → 5 * factorial(4) → 4 * factorial(3) → 3 *  
factorial(2)  
           → 2 * factorial(1) → 1 * factorial(0) → 1  
           ⇒ 120
```

⚠️ **Go does not do tail-call optimisation.** Deep recursion grows the stack. Goroutine stacks start at 8 KB and grow dynamically to a 1 GB default limit — so you get much further than in C, but a runaway recursion ends in `fatal error: stack overflow`, which is unrecoverable.

📌 The iterative version, which the course leaves commented out, is what you'd ship:

```
result := 1  
for i := 1; i <= number; i++ {  
    result = result * i  
}  
return result
```

Recursion earns its place on genuinely recursive structures — trees, filesystem walks, JSON traversal, parsers. Not on loops-in-disguise.

⚠️ `factorial(21)` silently overflows `int64`. Go has no overflow panic — it wraps. Use `math/big` for real arbitrary precision.

8.5 Variadic functions

```
func sumup(startingValue int, numbers ...int) int {
    sum := 0
    for _, val := range numbers {
        sum += val
    }
    return sum
}

sum := sumup(1, 10, 15, 40, -5)

numbers := []int{1, 10, 15}
anotherSum := sumup(1, numbers...) // 📌 spread an
existing slice
```

📌 Rules:

- The variadic parameter must be **last**, and there can only be one
- Inside the function it is a `[]T` — `len`, `range`, indexing all work
- Calling with no variadic args gives a **nil slice** (not empty), which is fine to range over
- `f(s...)` passes `s` directly — **no copy is made**, so the function can mutate the caller's slice

⚠️ The course example has a real bug worth catching: `startingValue` is never added to `sum`.

`sumup(1, 10, 15, 40, -5)` returns 60, not 61.

⚠️ `f(a, b, c)` and `f(s...)` cannot be mixed. To combine, build the slice first:

```
all := append([]int{extra}, s...)
f(all...)
```

🔧 The stdlib is full of them — `fmt.Println(a ...any)`, `append(s []T, v ...T)`,
`errors.Join(errs ...error)`, `slog.Info(msg string, args ...any)`.

8.6 `defer` — introduced here in the course's S10, but it belongs with functions

```
func (fm FileManager) ReadLines() ([]string, error) {
    file, err := os.Open(fm.InputFilePath)
    if err != nil {
        return nil, errors.New("failed to open file")
    }
    defer file.Close()           // 📌 runs when ReadLines
                                // returns, on EVERY path
    // ... every early return below is now safe
}
```

📌 The rules, in order of how often they matter:

1. Runs when the **surrounding function** returns — not at end of block, not at end of loop
2. **LIFO order** — multiple defers run last-registered-first
3. **Arguments are evaluated at `defer` time**, the call happens later
4. Runs even if the function **panics** — this is what makes cleanup reliable
5. Can **modify named return values**

```
// Rule 3 in action:
i := 0
defer fmt.Println(i) // prints 0 — i was captured NOW
i = 1
// vs.
defer func() { fmt.Println(i) }() // prints 1 — closure
reads i LATER
```

⚠️ **defer** in a loop accumulates until the function returns — a classic fd leak:

```
for _, path := range paths {
    f, _ := os.Open(path)
    defer f.Close() // ✗ all files stay open until
the function ends
}
// ✓ extract the body into its own function so each defer
fires per iteration
for _, path := range paths {
    if err := processFile(path); err != nil { return err }
}
```

⚠️ **defer f.Close()** swallows the error. For writes that's data loss:

```
func write(path string, data []byte) (err error) {
    f, err := os.Create(path)
    if err != nil { return err }
    defer func() {
        if cerr := f.Close(); cerr != nil && err == nil {
            err = cerr // 📌 named return lets defer
report the close failure
        }
    }()
    _, err = f.Write(data)
    return err
}
```

recover — only meaningful inside a deferred function:

```
func safeCall(f func()) (err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("recovered: %v", r)
        }
    }()
    f()
    return nil
}
```

🔧 Legitimate uses are narrow: an HTTP middleware that turns a handler panic into a 500 rather than killing the process (`gin.Default()` includes one), and recursive-descent parsers that use panic for control flow internally. **recover** cannot catch **fatal error:** — concurrent map writes, stack

overflow, and OOM all kill the process regardless.

S09 · Project: Price Calculator

16 lectures · ~1h 28m · repo: `code/09-practice-prj-price-calculator/01...11`

The synthesis project. A tool that reads prices from a file, applies a set of tax rates, writes JSON results. Its real subject is **dependency inversion via interfaces** — the whole build is a march from hardcoded file access to a swappable I/O layer.

9.1 Package structure

```

price-calculator/
├─ main.go                    # wiring only
├─ prices/prices.go          # domain logic – the
job
├─ conversion/conversion.go  # []string → []float64
├─ iomanager/iomanager.go    # the interface (2
lines, most important file)
├─ filemanager/filemanager.go # implementation: files
└─ cmdmanager/cmdmanager.go  # implementation:
stdin/stdout

```

 Each package is one responsibility, and the interface package has **no dependencies at all**.

9.2 The interface that makes it work

```

package iomanager

type IOManager interface {
    ReadLines() ([]string, error)
    WriteResult(data interface{}) error //  `any` in
modern Go
}

```

Two methods, its own package, imported by `prices` — never by the implementations. `filemanager` and `cmdmanager` don't import `iomanager` at all; they just happen to have matching methods.

 **That's implicit satisfaction paying rent.** Adding an S3 manager or an HTTP manager means writing a new package and changing one line in `main.go`. No existing file is touched.

9.3 Two implementations, one shape

```

// filemanager - production path
type FileManager struct {
    InputFilePath  string
    OutputFilePath string
}

func (fm FileManager) ReadLines() ([]string, error) {
    file, err := os.Open(fm.InputFilePath)
    if err != nil {
        return nil, errors.New("failed to open file")
    }
    defer file.Close()

    scanner := bufio.NewScanner(file)
    var lines []string
    for scanner.Scan() {
        lines = append(lines, scanner.Text())
    }
    if err := scanner.Err(); err != nil {
        return nil, errors.New("failed to read line in file")
    }
    return lines, nil
}

func (fm FileManager) WriteResult(data interface{}) error {
    file, err := os.Create(fm.OutputFilePath)
    if err != nil {
        return errors.New("failed to create file")
    }
    defer file.Close()

    encoder := json.NewEncoder(file)
    if err := encoder.Encode(data); err != nil {
        return errors.New("failed to convert data to JSON")
    }
    return nil
}

```

```
func New(inputPath, outputPath string) FileManager {
    return FileManager{InputFilePath: inputPath,
OutputFilePath: outputPath}
}
```

```
// cmdmanager — interactive path, same interface
type CMDManager struct{}

func (cmd CMDManager) ReadLines() ([]string, error) {
    fmt.Println("Please enter your prices. Confirm every
price with ENTER")
    var prices []string
    for {
        var price string
        fmt.Print("Price: ")
        fmt.Scan(&price)
        if price == "0" {
            break
        }
        prices = append(prices, price)
    }
    return prices, nil
}

func (cmd CMDManager) WriteResult(data interface{}) error
{
    fmt.Println(data)
    return nil
}

func New() CMDManager { return CMDManager{} }
```

 `CMDManager` is an **empty struct** — it holds no state, it only exists to hang methods on.

`struct{}` occupies zero bytes.

⚠️ `CMDManager.ReadLines` returns `nil` for the error unconditionally. Honest signature-wise (it satisfies the interface), but it means a `fmt.Scan` failure vanishes. Interfaces force a shared shape even when an implementation has nothing to say.

9.4 The job — pointer receivers and stored dependencies

```
package prices

type TaxIncludedPriceJob struct {
    IOManager          iomanager.IOManager `json:"-"`           //
    ⚠️ don't marshal the dependency
    TaxRate             float64              `json:"tax_rate"`
    InputPrices         []float64
    `json:"input_prices"`
    TaxIncludedPrices map[string]string
    `json:"tax_included_prices"`
}

func (job *TaxIncludedPriceJob) LoadData() error {
    lines, err := job.IOManager.ReadLines()
    if err != nil {
        return err
    }
    prices, err := conversion.StringsToFloats(lines)
    if err != nil {
        return err
    }
    job.InputPrices = prices // ← mutation: needs the
    pointer receiver
    return nil
}

func (job *TaxIncludedPriceJob) Process() error {
    if err := job.LoadData(); err != nil {
```

```

        return err
    }

    result := make(map[string]string)
    for _, price := range job.InputPrices {
        taxIncludedPrice := price * (1 + job.TaxRate)
        result[fmt.Sprintf("%.2f", price)] =
fmt.Sprintf("%.2f", taxIncludedPrice)
    }

    job.TaxIncludedPrices = result
    return job.IOManager.WriteResult(job)
}

func NewTaxIncludedPriceJob(iom iomanager.IOManager,
taxRate float64) *TaxIncludedPriceJob {
    return &TaxIncludedPriceJob{
        IOManager:    iom,
        InputPrices: []float64{10, 20, 30}, // defaults,
overwritten by LoadData
        TaxRate:      taxRate,
    }
}

```

 **json:"-"** on **IOManager** is load-bearing. `WriteResult(job)` marshals the job itself; without the tag, `encoding/json` tries to serialise an interface holding a **FileManager** and you'd emit the file paths into the results file.

 **The constructor takes the interface, not a concrete type.** That's dependency injection with no framework:

```

fm := filemanager.New("prices.txt",
fmt.Sprintf("result_%.0f.json", taxRate*100))
// cmdm := cmdmanager.New()           ← one-line swap
priceJob := prices.NewTaxIncludedPriceJob(fm, taxRate)
err := priceJob.Process()

```

9.5 Conversion helper

```

func StringsToFloats(strings []string) ([]float64, error)
{
    var floats []float64
    for _, stringVal := range strings {
        floatVal, err := strconv.ParseFloat(stringVal, 64)
        if err != nil {
            return nil, errors.New("failed to convert string to
float")
        }
        floats = append(floats, floatVal)
    }
    return floats, nil
}

```

 Two improvements you'd make in review:

```

func StringsToFloats(in []string) ([]float64, error) {
    out := make([]float64, 0, len(in))           // 1.
    preallocate
    for i, s := range in {
        v, err := strconv.ParseFloat(strings.TrimSpace(s), 64)
        if err != nil {
            return nil, fmt.Errorf("line %d: parsing %q: %w",
i+1, s, err)    // 2. wrap + locate
        }
        out = append(out, v)
    }
    return out, nil
}

```

⚠️ Also note the parameter is named `strings` — which **shadows the** `strings` package. Same class of bug as `json := json.Marshal(...)` in S05.

9.6 `main.go`

```
func main() {
    taxRates := []float64{0, 0.07, 0.1, 0.15}

    for _, taxRate := range taxRates {
        fm := filemanager.New("prices.txt",
            fmt.Sprintf("result_%.0f.json", taxRate*100))
        priceJob := prices.NewTaxIncludedPriceJob(fm, taxRate)
        if err := priceJob.Process(); err != nil {
            fmt.Println("Could not process job")
            fmt.Println(err)
        }
    }
}
```

Sequential: four rates, four files, one after another. **S10 makes this concurrent** — which is exactly why the course built it this way first.

9.7 What this project is actually teaching

📌 The refactor arc, lecture by lecture, is the lesson:

STEP	CHANGE	PRINCIPLE
02– 03	logic in <code>main</code> → a method	behaviour belongs with data
04– 06	file access → <code>filemanager</code> package	separate I/O from domain
07– 09	JSON output + struct tags	serialisation is a boundary concern
10	<code>IOManager</code> interface	depend on behaviour, not implementation
11	errors returned all the way up	failure is part of the contract

🔧 What's missing that you'd add for production: `context.Context` on `Process` for cancellation, `log/slog` instead of `fmt.Println`, wrapped errors, table-driven tests (trivial now — implement `IOManager` with an in-memory fake), and `float64` replaced by integer cents or `shopspring/decimal`,

because `0.1 + 0.2 != 0.3` and money in floats is a genuine outage waiting to happen.

S10 · Concurrency

10 lectures · ~42 min · repo: `code/10-concurrency/01...06`

📌 **Concurrency is not parallelism.** Concurrency is *structuring* a program as independently executing pieces; parallelism is *running* them simultaneously. Go gives you concurrency; the scheduler and your CPU count decide the parallelism.

10.1 The problem

```
func greet(phrase string) { fmt.Println("Hello!", phrase)
}

func slowGreet(phrase string) {
    time.Sleep(3 * time.Second)    // simulate slow work
    fmt.Println("Hello!", phrase)
}

func main() {
    greet("Nice to meet you!")
    greet("How are you?")
    slowGreet("How ... are ... you ...?")    // ← everything
below waits 3s
    greet("I hope you're liking the course!")
}
```

Sequential. Total time $\approx 3s$, and the last greeting is blocked by work it has nothing to do with.

10.2 Goroutines

```
go greet("Nice to meet you!")
go slowGreet("How ... are ... you ...?")
```

`go` before any call runs it in a new goroutine and returns immediately.

 **A goroutine is not an OS thread.** It's a green thread the Go runtime multiplexes onto a small pool of OS threads (the M:N scheduler). Start cost is ~ 2 KB of stack and a few hundred nanoseconds — spawning 100,000 is normal; spawning 100,000 OS threads would kill the machine. The stack grows and shrinks dynamically, so you don't pre-size it.

⚠️ **main** returning kills every goroutine instantly. No cleanup, no defer, no warning:

```
func main() {  
    go greet("Hello")    // probably never prints  
}                        // main returns → process exits
```

📌 **You must synchronise.** `time.Sleep` "works" and is always wrong — it's a race with extra steps.

10.3 Channels

```
done := make(chan bool)           // unbuffered  
  
func greet(phrase string, doneChan chan bool) {  
    fmt.Println("Hello!", phrase)  
    doneChan <- true               // send  
}  
  
func main() {  
    done := make(chan bool)  
    go greet("Nice to meet you!", done)  
    <-done                        // receive — BLOCKS until a  
    value arrives  
}
```

📌 Operators: `ch <- v` sends, `v := <-ch` receives, `close(ch)` closes.

🧠 **An unbuffered channel is a synchronisation point, not a queue.** The send blocks until a receiver is ready and vice versa — the two goroutines meet, hand over the value, and both continue. That rendezvous is the happens-before guarantee.

```
ch := make(chan int)      // unbuffered: send blocks until
                           received
ch := make(chan int, 5)   // buffered: send blocks only
                           when 5 are queued
```

Receiving with comma-ok distinguishes a real value from a closed channel:

```
v, ok := <-ch    // ok == false → channel closed AND
                  drained; v is the zero value
```

Range over a channel — receives until closed:

```
for v := range ch {
    // ...
} // loop ends when someone calls close(ch)
```

⚠ The course's multi-channel example is a **deliberate deadlock demo**:

```
done := make(chan bool)
go greet("Nice to meet you!", done)
go greet("How are you?", done)
go slowGreet("How ... are ... you ...?", done) // this
one calls close(done)
go greet("I hope you're liking the course!", done)

for range done { } // 💥 fatal error: all goroutines
are asleep - deadlock!
```

The `close` happens in `slowGreet` after 3 seconds, but the three fast goroutines send and exit first; whether the loop drains cleanly or blocks depends on scheduling. 📌 **The lesson:** `close` belongs to the sender, and with multiple senders you cannot safely close from inside any one of them (a send on a closed channel panics). Use `sync.WaitGroup` + a single closer goroutine.

Channel axioms — memorise these six:

OPERATION	NIL CHANNEL	OPEN, EMPTY	OPEN, FULL	CLOSED
send	blocks forever	blocks (unbuf) / queues	blocks	panics
receive	blocks forever	blocks	receives	zero value, <code>ok=false</code> , immediately
close	panics	ok	ok	panics

1. Send on closed → panic. Close of closed → panic. Close of nil → panic.
2. Receive from closed → returns immediately with the zero value, forever.
3. **Only the sender closes**, and only when there's exactly one sender.
4. You don't have to close a channel — GC handles it. Close only to *signal* "no more values."
5. A nil channel blocks forever, which is genuinely useful in `select` (disable a case by nil-ing it).
6. Unbuffered = synchronous handoff. Buffered = asynchronous up to cap.

Directional channel types — document intent and let the compiler enforce it:

```
func produce(out chan<- int) { out <- 1 } // send-only
func consume(in <-chan int)  { <-in }    // receive-
only
```

 Use these in every function signature that takes a channel. Free correctness.

10.4 `select`

Waits on multiple channel operations; takes whichever is ready first, at random if several are:

```
select {
case err := <-errorChan:
    if err != nil {
        fmt.Println(err)
    }
case <-doneChan:
    fmt.Println("Done!")
}
```

Non-blocking with `default:`:

```
select {
case v := <-ch:
    use(v)
default:
    // nothing ready — don't block
}
```

Timeout — the pattern you'll use constantly:

```
select {
case result := <-work:
    return result, nil
case <-time.After(5 * time.Second):
    return nil, errors.New("timeout")
case <-ctx.Done():
    return nil, ctx.Err()
}
```

⚠ `select {}` with no cases blocks forever — occasionally used to park `main`.

⚠ `time.After` allocates a timer that lives until it fires. In a hot loop, use `time.NewTimer` and `defer t.Stop()`.

10.5 The concurrent price calculator

`Process` changes from returning an error to *sending* on channels — the signature shift is the lesson:

```
func (job *TaxIncludedPriceJob) Process(doneChan chan
bool, errorChan chan error) {
    err := job.LoadData()
    if err != nil {
        errorChan <- err
        return // 📌 must return —
    }
    // otherwise it also sends on doneChan

    result := make(map[string]string)
    for _, price := range job.InputPrices {
        taxIncludedPrice := price * (1 + job.TaxRate)
        result[fmt.Sprintf("%.2f", price)] =
fmt.Sprintf("%.2f", taxIncludedPrice)
    }

    job.TaxIncludedPrices = result
    job.IOManager.WriteResult(job)
    doneChan <- true
}
```

```
func main() {
    taxRates := []float64{0, 0.07, 0.1, 0.15}
    doneChans := make([]chan bool, len(taxRates))
    errorChans := make([]chan error, len(taxRates))

    for index, taxRate := range taxRates {
        doneChans[index] = make(chan bool)
        errorChans[index] = make(chan error)
        fm := filemanager.New("prices.txt",
fmt.Sprintf("result_%.0f.json", taxRate*100))
        priceJob := prices.NewTaxIncludedPriceJob(fm, taxRate)
    }
```

```

    go priceJob.Process(doneChans[index],
errorChans[index])
}

for index := range taxRates {
    select {
    case err := <-errorChans[index]:
        if err != nil { fmt.Println(err) }
    case <-doneChans[index]:
        fmt.Println("Done!")
    }
}
}

```

Four jobs now run concurrently; with a 3s sleep in `WriteResult`, total time drops from ~12s to ~3s.

⚠ **Real problems with this code**, all worth naming:

- **A channel pair per job** is heavy. One shared `chan error` plus a `WaitGroup` is the idiom.
- `WriteResult`'s **error is discarded** — the return value isn't checked.
- The second loop consumes channels **in index order**, so a fast job 3 waits behind a slow job 0. You get concurrent *execution* but serialised *reporting*.
- **Unbuffered channels + an early return**: if `main` stopped receiving, every worker would block forever. Goroutine leak.

🔧 **How you'd actually write it:**

```

func main() {
    taxRates := []float64{0, 0.07, 0.1, 0.15}

    var wg sync.WaitGroup

```

```

    errCh := make(chan error, len(taxRates)) // 📌
buffered = senders never block

    for _, taxRate := range taxRates {
        wg.Add(1)
        go func(rate float64) { // Go 1.22+:
`go func()` capturing taxRate is fine
            defer wg.Done() // 📌 defer, so
a panic still decrements
                fm := filemanager.New("prices.txt",
fmt.Sprintf("result_%.0f.json", rate*100))
                if err := prices.NewTaxIncludedPriceJob(fm,
rate).Process(); err != nil {
                    errCh <- fmt.Errorf("rate %.2f: %w", rate, err)
                }
            }(taxRate)
        }

        wg.Wait()
        close(errCh) // 📌 safe:
one closer, after all senders are done

        for err := range errCh {
            fmt.Println(err)
        }
    }
}

```

Or with `errgroup`, which cancels siblings on the first failure:

```

g, ctx := errgroup.WithContext(context.Background())
g.SetLimit(8) //
bounded concurrency
for _, rate := range taxRates {
    g.Go(func() error { return process(ctx, rate) })
}
if err := g.Wait(); err != nil { ... }

```

10.6 defer (covered here in the course)

See **S08.6** for the full treatment. The course introduces it to fix the file-handle cleanup in

`filemanager`:

```
file, err := os.Open(fm.InputFilePath)
if err != nil { return nil, errors.New("failed to open
file") }
defer file.Close()      // replaces three scattered
file.Close() calls
```

 The win: every early `return` below is automatically safe, and you can't forget one when adding a new error branch later.

10.7 The concurrency toolbox beyond the course

 The course covers goroutines, channels, select, defer. This is the rest of what you need:

```
// sync.WaitGroup — wait for N goroutines
var wg sync.WaitGroup
wg.Add(1)
go func() { defer wg.Done(); work() }()
wg.Wait()

// sync.Mutex — protect shared state
type Counter struct {
    mu sync.Mutex
    n   int
}
func (c *Counter) Inc() { c.mu.Lock(); defer
c.mu.Unlock(); c.n++ }
//  sync.RWMutex when reads vastly outnumber writes

// sync.Once — exactly-once initialisation
```

```
var once sync.Once
once.Do(func() { initExpensiveThing() })

// atomic — lock-free counters
var hits atomic.Int64
hits.Add(1)
```

 **context.Context** — the single most important thing the course omits.

Every I/O-bound function in
a real service takes one as its first parameter:

```
func FetchUser(ctx context.Context, id int64) (*User,
error) {
    ctx, cancel := context.WithTimeout(ctx, 2*time.Second)
    defer cancel() //  ALWAYS defer
    cancel — otherwise the timer leaks

    row := db.QueryRowContext(ctx, "SELECT ... WHERE id =
?", id)
    // ...
}
```

Cancellation propagates down the tree: client disconnects → `ctx.Done()`
closes → the DB query, the
HTTP call, and every worker below it all abort. Without it, a cancelled request
keeps burning resources.

 **The race detector.** Run it in CI, always:

```
go test -race ./...
go run -race .
```

It catches unsynchronised access at runtime with ~10× slowdown. Data races in
Go are undefined
behaviour, not "eventually consistent" — a torn slice header can produce
memory corruption.

 **Goroutine leaks** are Go's memory leak. A goroutine blocked forever on a channel is never collected.

Watch `runtime.NumGoroutine()`, or scrape `/debug/pprof/goroutine`. Rules that prevent them:

1. Every goroutine needs a guaranteed exit path — a closed channel, a cancelled context, or a finite loop
2. The goroutine that starts something owns stopping it
3. Never send on a channel nobody is guaranteed to read

 **"Don't communicate by sharing memory; share memory by communicating."** — but the corollary matters too: **a mutex is often the simpler, faster answer** for guarding a struct field. Channels are for transferring ownership and coordinating; mutexes are for protecting state. Reach for the mutex when the

answer is "I have a map several goroutines touch."

S11 · Project: Event Booking REST API

repo: `code/11-rest-api/01...16` · *Gin* + *SQLite* + *JWT* + *bcrypt*

The capstone. Every concept from S02–S10 lands here in a shape you'd recognise from a real service.

11.1 The API surface

METHOD	ROUTE	AUTH	NOTES
GET	/events	—	list
GET	/events/:id	—	single
POST	/events	✓ JWT	creator recorded
PUT	/events/:id	✓ + owner	
DELETE	/events/:id	✓ + owner	
POST	/signup	—	bcrypt hash
POST	/login	—	returns JWT
POST	/events/:id/register	✓ JWT	
DELETE	/events/:id/register	✓ JWT	

📌 The distinction between **authentication** (valid token → middleware) and **authorization**

(`event.UserID == userId` → handler) is drawn cleanly, and it's the part most tutorials conflate.

11.2 Layout

```
rest-api/
├─ main.go           # 8 lines: init DB, create
server, register routes, run
├─ db/db.go          # connection + schema
├─ models/           # Event, User – struct +
persistence methods
├─ routes/           # HTTP handlers + route table
├─ middleware/auth.go # JWT gate
└─ utils/            # hash.go, jwt.go
```

🔧 This is a clean starter layout. For production, move everything except `main.go` under `internal/` and split `models` into `domain` (structs) + `store` (SQL) so the domain has no DB import.

11.3 main.go

```
func main() {
    db.InitDB()
    server := gin.Default()           // Logger + Recovery
    middleware pre-attached
    routes.RegisterRoutes(server)
    server.Run(":8080")
}
```

11.4 Database

```
package db

import (
    "database/sql"
    _ "github.com/mattn/go-sqlite3"    // 📌 blank import:
    registers the driver via init()
)

var DB *sql.DB

func InitDB() {
    var err error
    DB, err = sql.Open("sqlite3", "api.db")
    if err != nil {
        panic("Could not connect to database.")
    }

    DB.SetMaxOpenConns(10)
    DB.SetMaxIdleConns(5)

    createTables()
}
```

 **sql.Open** does not connect. It validates arguments and prepares a lazy connection pool; the first real connection happens on first use. To fail fast at startup, call `DB.Ping()`. This surprises everyone once.

 ***sql.DB** is a pool, not a connection. It's goroutine-safe and meant to be long-lived — create one per process, never per request. Pool settings that matter:

```
DB.SetMaxOpenConns(25)           // cap — must be
                                  ≤ your DB's max_connections / replicas
DB.SetMaxIdleConns(25)           // keep warm; set
                                  equal to MaxOpenConns for steady load
DB.SetConnMaxLifetime(5 * time.Minute) //  the course
                                  omits this — needed behind a load balancer
```

Schema with `IF NOT EXISTS` + foreign keys:

```
createEventsTable := `
CREATE TABLE IF NOT EXISTS events (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  description TEXT NOT NULL,
  location TEXT NOT NULL,
  dateTime DATETIME NOT NULL,
  user_id INTEGER,
  FOREIGN KEY(user_id) REFERENCES users(id)
)`
if _, err := DB.Exec(createEventsTable); err != nil {
  panic("Could not create events table.")
}
```

 Three things to fix before this is production code:

1. A global `var DB *sql.DB` makes the package untestable and hides

the dependency. Inject a

`*sql.DB` (or a `Store` interface) into your handlers instead.

2. **Schema-in-code doesn't version.** Use `golang-migrate` or `goose` — you need reversible, ordered migrations the moment a second person touches the DB.

3. **SQLite doesn't enforce foreign keys by default.** Needs `PRAGMA foreign_keys = ON;` per connection.

11.5 Models — struct + persistence, with `binding` tags

```
type Event struct {
    ID          int64
    Name        string    `binding:"required"`           // Gin
validation
    Description string    `binding:"required"`
    Location   string    `binding:"required"`
    DateTime   time.Time `binding:"required"`
    UserID     int64
}

func (e *Event) Save() error {
    query := `
        INSERT INTO events(name, description, location,
dateTime, user_id)
        VALUES (?, ?, ?, ?, ?)`
    stmt, err := db.DB.Prepare(query)
    if err != nil {
        return err
    }
    defer stmt.Close()                                // 📌 prepared
statements MUST be closed

    result, err := stmt.Exec(e.Name, e.Description,
e.Location, e.DateTime, e.UserID)
    if err != nil {
        return err
    }
}
```

```

}
id, err := result.LastInsertId()
e.ID = id // pointer
receiver: writes the ID back
return err
}

```

 **? placeholders are the SQL injection defence.** The driver sends query and parameters separately — the value is never parsed as SQL. String-concatenating a query is the vulnerability; there is no "escaping" alternative in Go, and there doesn't need to be.

Query patterns:

```

// Many rows
rows, err := db.DB.Query("SELECT * FROM events")
if err != nil { return nil, err }
defer rows.Close() //  leaks a
connection if you forget

var events []Event
for rows.Next() {
    var event Event
    if err := rows.Scan(&event.ID, &event.Name,
&event.Description,
    &event.Location, &event.DateTime, &event.UserID); err
!= nil {
        return nil, err
    }
    events = append(events, event)
}
return events, rows.Err() //  course omits
rows.Err() — same trap as scanner.Err()

// One row — no Close needed, QueryRow defers the error to
Scan

```

```

row := db.DB.QueryRow("SELECT * FROM events WHERE id = ?",
id)
var event Event
if err := row.Scan(&event.ID, ...); err != nil {
    return nil, err // 🔧 check errors.Is(err,
sql.ErrNoRows) → 404, not 500
}

```

⚠️ **SELECT *** couples your Scan argument order to the physical column order. Add a column and every Scan breaks at runtime with a type mismatch. Always list columns explicitly.

⚠️ Prepare + Exec + Close for a single execution is three round trips.

```
db.Exec(query, args...)
```

does it in one. Prepared statements pay off when reused in a loop.

11.6 Handlers

```

func createEvent(context *gin.Context) {
    var event models.Event
    if err := context.ShouldBindJSON(&event); err != nil {
        context.JSON(http.StatusBadRequest, gin.H{"message":
"Could not parse request data."})
        return
    }

    userId := context.GetInt64("userId") // set by the
auth middleware
    event.UserID = userId

    if err := event.Save(); err != nil {
        context.JSON(http.StatusInternalServerError,
gin.H{"message": "Could not create event."})
        return
    }
}

```

```
context.JSON(http.StatusCreated, gin.H{"message": "Event
created!", "event": event})
}
```

- `ShouldBindJSON` unmarshals **and** runs `binding:"required"` validation
- `gin.H` is just `map[string]any`
- `context.Param("id")` → path params, then `strconv.ParseInt(s, 10, 64)`

Ownership check — authorization, in the handler:

```
event, err := models.GetEventByID(eventId)
if err != nil {
    context.JSON(http.StatusInternalServerError,
gin.H{"message": "Could not fetch the event."})
    return
}
if event.UserID != userId {
    context.JSON(http.StatusUnauthorized, gin.H{"message":
"Not authorized to update event."})
    return
}
```

 Correct status is **403 Forbidden** — 401 means "you didn't authenticate", 403 means "you did, and you still can't." The course uses 401 for both.

11.7 Password hashing

```

package utils

import "golang.org/x/crypto/bcrypt"

func HashPassword(password string) (string, error) {
    bytes, err :=
    bcrypt.GenerateFromPassword([]byte(password), 14)
    return string(bytes), err
}

func CheckPasswordHash(password, hashedPassword string)
bool {
    err :=
    bcrypt.CompareHashAndPassword([]byte(hashedPassword),
    []byte(password))
    return err == nil
}

```

📌 **bcrypt is deliberately slow** — cost 14 means 2^{14} iterations, ~1 second on typical hardware. That's the feature: it caps an attacker's guess rate. It also caps your login throughput, so 10–12 is the more common production setting. Benchmark on your actual hardware.

📌 The salt is generated per-hash and **stored inside the output string** — that's why `CompareHashAndPassword` needs no salt argument, and why you never store one separately.

⚠️ `CompareHashAndPassword` is constant-time. Never compare hashes with `==`; that's a timing oracle.

⚠️ bcrypt **silently truncates input at 72 bytes**. For long passphrases, SHA-256 first or use Argon2id (`golang.org/x/crypto/argon2`), which is the current recommendation for new systems.

11.8 JWT

```
const secretKey = "supersecret" // ⚠️ see below

func GenerateToken(email string, userId int64) (string,
error) {
    token := jwt.NewWithClaims(jwt.SigningMethodHS256,
jwt.MapClaims{
        "email": email,
        "userId": userId,
        "exp":    time.Now().Add(time.Hour * 2).Unix(),
    })
    return token.SignedString([]byte(secretKey))
}

func VerifyToken(token string) (int64, error) {
    parsedToken, err := jwt.Parse(token, func(token
*jwt.Token) (interface{}, error) {
        _, ok := token.Method.(*jwt.SigningMethodHMAC)
        if !ok {
            return nil, errors.New("unexpected signing method")
        }
        // 📌 critical — see below
        return []byte(secretKey), nil
    })
    if err != nil {
        return 0, errors.New("could not parse token")
    }
    if !parsedToken.Valid {
        return 0, errors.New("invalid token")
    }

    claims, ok := parsedToken.Claims.(jwt.MapClaims)
    if !ok {
        return 0, errors.New("invalid token claims")
    }
}
```

```

    userID := int64(claims["userID"].(float64))    // 📌
    JSON numbers are float64
    return userID, nil
}

```

📌 **The signing-method check is a real vulnerability defence, not boilerplate.** Without it, an attacker sends a token with `"alg": "none"` (or swaps HMAC for RSA and signs with your public key) and the library validates it. This was a widespread CVE class across JWT libraries. Always assert the method.

📌 `claims["userID"].(float64)` — `MapClaims` is `map[string]any` populated by `encoding/json`, and JSON has one number type. Asserting `.(int64)` panics. Use a typed claims struct instead:

```

type Claims struct {
    UserID int64   `json:"userID"`
    Email  string `json:"email"`
    jwt.RegisteredClaims // embeds exp,
    iat, iss, sub
}

token, err := jwt.ParseWithClaims(tokenStr, &Claims{},
    keyFunc)

```

⚠️ Everything wrong with the course's JWT setup, ranked:

1. **Hardcoded secret in source.** Must come from env/secret manager. It's in git forever otherwise.
2. **A short string secret.** HS256 needs ≥ 32 bytes of entropy; `"supersecret"` is brute-forceable.
3. **No Bearer prefix** — the header is read raw. The standard is `Authorization: Bearer <token>`.
4. **JWTs can't be revoked.** A stolen token is valid until `exp`. Real

systems pair a short-lived
access token (5–15 min) with a revocable refresh token.

5. `fmt.Println("Could not parse token")` inside a util — log with `slog`, don't print.

 **A JWT is signed, not encrypted.** Anyone can base64-decode the payload and read it. Never put anything secret in claims.

11.9 Middleware

```
func Authenticate(context *gin.Context) {  
    token := context.Request.Header.Get("Authorization")  
    if token == "" {  
        context.AbortWithStatusJSON(http.StatusUnauthorized,  
            gin.H{"message": "Not authorized."})  
        return  
    }  
  
    userId, err := utils.VerifyToken(token)  
    if err != nil {  
        context.AbortWithStatusJSON(http.StatusUnauthorized,  
            gin.H{"message": "Not authorized."})  
        return  
    }  
  
    context.Set("userId", userId)           //  pass data down  
    the chain  
    context.Next()                          // hand off to the  
    next handler  
}
```

 **Abort* VS return:** a bare `return` exits *this* middleware but the chain continues to the handler. `AbortWithStatusJSON` stops the chain. Getting this wrong means an unauthenticated request

reaching your handler with a 401 already written — a real auth bypass.

Route groups:

```
func RegisterRoutes(server *gin.Engine) {
    server.GET("/events", getEvents)
    server.GET("/events/:id", getEvent)

    authenticated := server.Group("/")
    authenticated.Use(middlewares.Authenticate) //
    applies to everything below
    authenticated.POST("/events", createEvent)
    authenticated.PUT("/events/:id", updateEvent)
    authenticated.DELETE("/events/:id", deleteEvent)
    authenticated.POST("/events/:id/register",
registerForEvent)
    authenticated.DELETE("/events/:id/register",
cancelRegistration)

    server.POST("/signup", signup)
    server.POST("/login", login)
}
```

11.10 Bugs in the course code worth spotting

Genuinely useful review practice — these are all in the final snapshot:

```
// 1. models/user.go — Save() has a VALUE receiver
func (u User) Save() error {
    // ...
    u.ID = userId // ! writes to the copy; caller's
User.ID stays 0
    return err
}
// Fix: func (u *User) Save() error
```

```

// 2. routes/register.go – err is assigned then
immediately overwritten
eventId, err := strconv.ParseInt(context.Param("id"), 10,
64)
var event models.Event
event.ID = eventId
err = event.CancelRegistration(userId) // ! the
ParseInt error is silently dropped

// 3. models/event.go – dead code
var events = []Event{} // leftover from
the in-memory version

// 4. updateEvent – no ownership re-check after binding,
and
//    updatedEvent.UserID is zero, so a subsequent read
shows no owner

// 5. User struct has no `json:"-"` on Password
type User struct {
    ID        int64
    Email     string `binding:"required"`
    Password  string `binding:"required"` // ! marshals
into responses
}

```

11.11 What you'd add for production

 The gap between this and a service you'd deploy:

MISSING	ADD
Config	env vars via <code>os.Getenv</code> / <code>caarlos0/env</code> ; never hardcode
Logging	<code>log/slog</code> structured JSON, request IDs, no <code>fmt.Println</code>
Context	<code>ctx</code> on every model method → <code>QueryRowContext</code> , <code>ExecContext</code>
Graceful shutdown	<code>signal.NotifyContext</code> + <code>srv.Shutdown(ctx)</code>
Timeouts	<code>http.Server{ReadTimeout, WriteTimeout, IdleTimeout}</code> — Gin's defaults are none
Migrations	<code>golang-migrate</code>
Tests	<code>httptest.NewRecorder()</code> + a <code>Store</code> interface with an in-memory fake
Rate limiting	on <code>/login</code> especially — <code>bcrypt</code> makes it a DoS vector
CORS	<code>gin-contrib/cors</code>
Pagination	<code>GET /events</code> returns the whole table today
Transactions	register + capacity check must be atomic (<code>db.BeginTx</code>)
Health check	<code>/healthz</code> hitting <code>DB.PingContext</code>
Observability	<code>/metrics</code> , <code>pprof</code> , OpenTelemetry traces

Graceful shutdown, since it's short and always needed:

```
func main() {
    srv := &http.Server{
        Addr:           ":8080",
        Handler:        router,
        ReadTimeout:    5 * time.Second,
        WriteTimeout:   10 * time.Second,
        IdleTimeout:    120 * time.Second,
    }

    go func() {
        if err := srv.ListenAndServe(); err != nil &&
            !errors.Is(err, http.ErrServerClosed) {
```

```

        log.Fatalf("listen: %v", err)
    }
}()

ctx, stop := signal.NotifyContext(context.Background(),
os.Interrupt, syscall.SIGTERM)
defer stop()
<-ctx.Done()

shutdownCtx, cancel :=
context.WithTimeout(context.Background(), 10*time.Second)
defer cancel()
if err := srv.Shutdown(shutdownCtx); err != nil {
    log.Printf("forced shutdown: %v", err)
}
}

```

 Without this, a Kubernetes rolling deploy drops every in-flight request on SIGTERM.

 **On Gin:** fine, fast, huge ecosystem. Also worth knowing `chi` (closest to `stdlib`, `http.Handler` all the way down) and `echo`. Since Go 1.22, `net/http`'s own mux does method and path-parameter routing — `mux.HandleFunc("GET /events/{id}", h)` — so for a small service you may not need a framework

at all.

S12 · Beyond the Course

The course closes with roundup and next steps. Here's the gap between "finished the course" and "comfortable in a Go codebase."

12.1 Testing — the biggest omission

The course has no testing section at all. Go's is built in and it's good.

```
// prices/prices_test.go — file must end in _test.go,
package can be `prices` or `prices_test`
package prices

import "testing"

func TestTaxCalculation(t *testing.T) {
    got := applyTax(100, 0.2)
    if got != 120 {
        t.Errorf("applyTax(100, 0.2) = %v; want 120", got)
    }
    // 📌 got/want phrasing is the convention
}
```

`t.Errorf` records a failure and continues; `t.Fatalf` stops the test immediately.

📌 **Table-driven tests are *the* Go testing idiom.** Learn this shape and use it everywhere:

```
func TestApplyTax(t *testing.T) {
    tests := []struct {
        name    string
        price   float64
        rate    float64
        want    float64
    }{
        {"no tax",      100, 0,      100},
        {"seven pct",   100, 0.07, 107},
        {"zero price",  0, 0.2,      0},
    }

    for _, tt := range tests {
```

```

    t.Run(tt.name, func(t *testing.T) {           // subtests:
named, isolated, individually runnable
        if got := applyTax(tt.price, tt.rate); got !=
tt.want {
            t.Errorf("applyTax(%v, %v) = %v; want %v",
tt.price, tt.rate, got, tt.want)
        }
    })
}
}

```

```

go test ./...                                # everything
go test -run TestApplyTax/no_tax             # one subtest
go test -v -race -cover ./...                # the CI invocation
go test -coverprofile=c.out ./... && go tool cover -
html=c.out
go test -bench=. -benchmem ./...            # benchmarks

```

Testing HTTP handlers — no server needed:

```

func TestGetEvents(t *testing.T) {
    req := httptest.NewRequest(http.MethodGet, "/events",
nil)
    w := httptest.NewRecorder()

    router.ServeHTTP(w, req)

    if w.Code != http.StatusOK {
        t.Fatalf("status = %d; want 200", w.Code)
    }
}

```

Other essentials: `t.Cleanup(fn)` for teardown, `t.Parallel()` for concurrency, `t.TempDir()` for scratch files, `testing.F` for fuzzing (`go test -fuzz=Fuzz`), and `TestMain` for package-level setup.

 `stretchr/testify` (`assert`/`require`) is near-universal in the wild, though the `stdlib` style above is what you'll see in Go's own source. `google/go-cmp` for deep struct comparison beats `reflect.DeepEqual`.

12.2 The `stdlib` you'll reach for daily

```
log/slog      //  structured logging, stdlib since 1.21
— use instead of `log` and fmt.Println
context       // cancellation & deadlines — first param
of every I/O function
time          // time.Duration arithmetic, tickers,
timers
strings       // Builder ( for concatenation in loops),
Split, Join, Contains, TrimSpace
strconv       // Atoi, Itoa, ParseFloat, FormatFloat,
Quote
errors        // Is, As, Join, New, Unwrap
sort / slices // slices.Sort, Contains, Index, Clone,
Max, BinarySearch
maps          // maps.Keys, maps.Values (iterators since
1.23)
encoding/json // Marshal, Unmarshal, Encoder, Decoder
net/http      // client and server, both production-grade
os / io       // ReadFile, Open, io.Copy,
io.Reader/Writer
regexp        //  RE2 — linear time, no backtracking,
no catastrophic blowup
sync          // Mutex, RWMutex, WaitGroup, Once, Pool
```

 **String concatenation in a loop is $O(n^2)$** — strings are immutable, so every `+=` copies:

```
var sb strings.Builder
for _, s := range items {
    sb.WriteString(s)
}
result := sb.String()
```



log/slog is what you should be writing today:

```
logger := slog.New(slog.NewJSONHandler(os.Stdout,
&slog.HandlerOptions{Level: slog.LevelInfo}))
slog.SetDefault(logger)

slog.Info("event created", "event_id", e.ID, "user_id",
e.UserID)
slog.Error("db write failed", "err", err, "query",
"insert_event")
```

12.3 Recent Go worth knowing

VERSION FEATURE

1.18	Generics , fuzzing, workspaces (<code>go.work</code>)
1.20	<code>errors.Join</code> , wrapping multiple errors
1.21	log/slog , <code>slices/maps</code> packages, built-in <code>min/max/clear</code> , <code>cmp.Ordered</code>
1.22	Per-iteration loop variables (kills the closure bug), <code>for i := range 10</code> , enhanced <code>net/http</code> routing
1.23	Range-over-func iterators , <code>iter</code> package, <code>unique</code> package
1.24	Generic type aliases, <code>go tool</code> directive, weak pointers, <code>os.Root</code>

⚠ Loop-variable semantics are gated on the `go` directive in `go.mod`, not the installed toolchain.

Bump `go 1.22` in your modules to get the fix.

12.4 Tooling

```
gofmt -l .                                # list unformatted files
(CI gate)
goimports -w .                            # format + fix imports
go vet ./...                              # stdlib static analysis
staticcheck ./...                         # 🛠️ the good third-party
linter
golangci-lint run                         # 🛠️ meta-linter, what
most teams run in CI

go test -race ./...                      # data race detector
go test -bench=. -benchmem               # benchmarks + allocation
counts
go tool pprof                            # CPU/heap profiling
go build -gcflags='-m'                   # escape analysis: what
heap-allocates and why

go mod tidy && go mod verify              # dependency hygiene
govulncheck ./...                        # 📌 scans your deps
against the Go vuln DB
```

📌 **Profiling in a live service** — add one import and you get CPU, heap, goroutine, block and mutex profiles over HTTP:

```
import _ "net/http/pprof"
go func() {
    log.Println(http.ListenAndServe("localhost:6060", nil)) }
()
// go tool pprof http://localhost:6060/debug/pprof/heap
```

⚠️ Never expose that port publicly.

12.5 Idiom checklist for code review

- ✓ Errors wrapped with `%w` and context; checked with `errors.Is/errors.As`, never `==` or string match
- ✓ `context.Context` is the first parameter of every I/O-bound function, named `ctx`
- ✓ Interfaces defined by the consumer, small, and returned as concrete types
- ✓ One receiver kind per type; pointer receivers if any method mutates
- ✓ Slices preallocated with `make([]T, 0, n)` where `n` is known
- ✓ Maps initialised before write; comma-ok on every read that can miss
- ✓ `defer` for every `Close`, and never inside a loop body
- ✓ Every goroutine has a guaranteed exit path
- ✓ `json:"-"` on every secret field
- ✓ SQL uses `?/$1` placeholders, columns listed explicitly, `rows.Err()` checked
- ✓ `go vet`, `staticcheck` and `go test -race` clean in CI
- ✓ No `panic` across a package boundary
- ✓ Exported identifiers have doc comments starting with the identifier name

12.6 Where to go next

- **Effective Go** and the **Go Code Review Comments** wiki — short, and still the canonical style guide
- **The Go Memory Model** — read it before writing anything lock-free
- **Go by Example** (gobyexample.com) — fastest lookup for "how do I write X"
- `google/uuid`, `spf13/cobra`, `jmoiron/sqlx`, `golang-migrate`, `stretchr/testify`, `golang.org/x/sync/errgroup` — the small set of near-universal third-party packages
- Read the stdlib source. `net/http`, `sync` and `encoding/json` are all approachable and are the best available demonstration of idiomatic Go.

Appendix A · Cheat sheet

// — Declarations

```
var x int           // zero value
var x int = 5
var x = 5           // inferred
x := 5              // short form, function scope only
const Pi = 3.14
type Celsius float64 // new type
type Alias = float64 // alias
```

// — Types

```
int int8..int64 uint..uint64 float32 float64 string bool
byte = uint8    rune = int32    any = interface{}
[3]int          // array (fixed, value semantics)
[]int           // slice (dynamic, header: ptr/len/cap)
map[string]int  // map (nil map: read ok, write panics)
chan int        // channel
func(int) error // func type
*int            // pointer
struct{ X int } // struct
interface{ M() } // interface
```

// — Control

```
if v, err := f(); err != nil { }
for i := 0; i < n; i++ { }
for cond { }
for { }
for i, v := range coll { }
for i := range 10 { } // 1.22+
switch x { case 1, 2: default: }
```

```

switch { case a > b: }
switch v := i.(type) { case int: }
goto / labels / break outer / continue outer

// — Functions

```

```

func f(a, b int) (int, error)
func f(a int) (result int, err error)           // named
returns
func f(nums ...int)                           // variadic
f(slice...)                                   // spread
func (t T) M()                                // value
receiver
func (t *T) M()                               // pointer
receiver
func F[T any](v T) T                          // generic
defer f()                                     // LIFO, args
eval'd now
go f()                                         // goroutine

// — Slices

```

```

s := []int{1, 2, 3}
s := make([]int, 0, 10)                       // len 0, cap
10
s = append(s, v)                             // MUST
reassign
s = append(s, other...)
copy(dst, src)                               // returns n
copied
s[1:3] s[1:] s[:2] s[:] s[1:3:4]              // last: caps
capacity
len(s) cap(s)
slices.Sort/Contains/Index/Clone/Reverse      // 1.21+

// — Maps

```

```

m := map[string]int{"a": 1}

```

```

m := make(map[string]int, 10)
v, ok := m["k"] // ALWAYS use
comma-ok
delete(m, "k")
clear(m) // 1.21+
for k, v := range m { } // RANDOM
order

// — Channels


---


ch := make(chan int) //
unbuffered: sync handoff
ch := make(chan int, 5) // buffered
ch <- v          v := <-ch      v, ok := <-ch
close(ch) // sender
only, once
for v := range ch { } // until
closed
chan<- int // send-only      <-chan int // receive-
only
select { case v := <-ch: ; case <-time.After(d): ;
default: }

// — Errors


---


errors.New("msg")
fmt.Errorf("context: %w", err) // wrap
errors.Is(err, ErrTarget) // match
through the chain
errors.As(err, &target) // extract
typed error
errors.Join(err1, err2) // 1.20+

// — Concurrency


---


var wg sync.WaitGroup; wg.Add(1); go func(){ defer
wg.Done() }(); wg.Wait()
var mu sync.Mutex; mu.Lock(); defer mu.Unlock()

```

```

var once sync.Once; once.Do(f)
ctx, cancel := context.WithTimeout(ctx, d); defer cancel()

// — fmt verbs

```

```

%v %+v %#v %T    %d %s %t %q    %.2f %8.2f    %w (Errorf
only)    %%

```

Appendix B · The gotcha table

#	GOTCHA	SYMPTOM	FIX
1	<code>go run main.go</code> on a multi-file package	<code>undefined: someFunc</code>	<code>go run .</code>
2	Value receiver mutation	changes silently lost	pointer receiver
3	Nil map write	<code>panic: assignment to entry in nil map</code>	<code>make(map...)</code> first
4	Map read of missing key	silent zero value	comma-ok
5	Map iteration order	flaky tests, non-deterministic output	sort the keys
6	append aliasing	parent slice mutated unexpectedly	<code>slices.Clone</code> / 3-index slice
7	<code>make([]T, n)</code> vs <code>make([]T, 0, n)</code>	leading zeros in output	use the 3-arg form
8	append without reassign	changes vanish	<code>s = append(s, v)</code>
9	Loop var capture (< 1.22)	goroutines all see the last value	bump <code>go.mod</code> to 1.22+

10	<code>defer</code> in a loop	fd exhaustion	extract a function
11	<code>defer f.Close()</code> on a writer	silent data loss	named return + check
12	Nil interface holding a nil pointer	<code>err != nil</code> when it "shouldn't" be	return literal <code>nil</code>
13	<code>*T</code> method not in <code>T</code> 's method set	"does not implement" compile error	use <code>&T{}</code>
14	Unexported fields + JSON	marshals to <code>{}</code>	capitalise + tag
15	Struct tag typo (<code>json: "x"</code>)	tag silently ignored	no space after colon
16	<code>sql.Open</code> doesn't connect	failures surface on first query	<code>DB.Ping()</code> at startup
17	Missing <code>rows.Close()</code>	connection pool exhaustion	<code>defer rows.Close()</code>
18	<code>scanner.Err()</code> / <code>rows.Err()</code> unchecked	silent truncation	always check
19	Unbuffered channel, no receiver	<code>fatal error: all goroutines are asleep</code>	buffer or WaitGroup
20	Send on closed channel	panic	only the sender closes
21	Concurrent map access	<code>fatal error: concurrent map writes</code> (unrecoverable)	mutex / <code>sync.Map</code>
22	Goroutine blocked forever	memory grows, <code>NumGoroutine</code> climbs	context cancellation
23	Integer division	<code>5/2 == 2</code>	convert to float first
24	<code>int(floatVar)</code> truncates; <code>int(9.99)</code>	9 instead of 10, or a build error	<code>math.Round</code> first

	on a constant won't compile		
25	<code>len(s)</code> on a UTF-8 string	counts bytes	<code>[]rune(s)</code> or <code>utf8.RuneCountInString</code>
26	Shadowing a package name (<code>json</code> , <code>strings</code>)	package unusable in that scope	rename the variable
27	<code>break</code> inside <code>switch</code> in a loop	breaks the switch only	labelled break
28	<code>range</code> value is a copy	mutations lost	index into the slice
29	Money in <code>float64</code>	rounding errors that compound	integer cents / decimal lib
30	<code>recover</code> in the wrong place	doesn't catch	must be in a deferred func

Appendix C · Standard library map

NEED	PACKAGE	KEY API
Formatted I/O	<code>fmt</code>	<code>Printf</code> , <code>Sprintf</code> , <code>Errorf</code> , <code>Scan</code>
Structured logs	<code>log/slog</code>	<code>Info</code> , <code>Error</code> , <code>NewJSONHandler</code>
Errors	<code>errors</code>	<code>New</code> , <code>Is</code> , <code>As</code> , <code>Join</code> , <code>Unwrap</code>
Files	<code>os</code>	<code>ReadFile</code> , <code>WriteFile</code> , <code>Open</code> , <code>Create</code>
Buffered I/O	<code>bufio</code>	<code>Scanner</code> , <code>Reader</code> , <code>Writer</code>
Streams	<code>io</code>	<code>Reader</code> , <code>Writer</code> , <code>Copy</code> , <code>ReadAll</code>
Paths	<code>path/filepath</code>	<code>Join</code> , <code>Base</code> , <code>Ext</code> , <code>WalkDir</code>
Strings	<code>strings</code>	<code>Builder</code> , <code>Split</code> , <code>Join</code> ,

		TrimSpace, ReplaceAll
Conversion	strconv	Atoi, Itoa, ParseFloat, Quote
Time	time	Now, Since, Duration, Ticker, After
JSON	encoding/json	Marshal, Unmarshal, Encoder, Decoder
HTTP	net/http	ListenAndServe, Client, ServeMux, Handler
HTTP testing	net/http/httptest	NewRecorder, NewRequest, NewServer
SQL	database/sql	Open, Query, QueryRow, Exec, BeginTx
Concurrency	sync	Mutex, RWMutex, WaitGroup, Once, Pool
Atomics	sync/atomic	Int64, Value, CompareAndSwap
Cancellation	context	WithCancel, WithTimeout, WithValue
Sorting	sort / slices	Slice, SortFunc, BinarySearch
Collections	slices / maps	Contains, Index, Clone, Keys, Values
Math	math	Round, Abs, Max, Inf, Pow
Randomness	math/rand/v2	IntN, Float64 (⚠ not crypto)
Crypto random	crypto/rand	Read — for tokens, salts, secrets
Hashing	crypto/sha256	Sum256
Passwords	golang.org/x/crypto/bcrypt	GenerateFromPassword, CompareHashAndPassword

Regex

	<code>regexp</code>	<code>MustCompile,</code> <code>FindStringSubmatch</code>
Testing	<code>testing</code>	<code>T, B, F, Run, Cleanup,</code> <code>TempDir</code>
Reflection	<code>reflect</code>	last resort — powerful, slow, unsafe

End of notes. Course: [Go — The Complete Guide](#) ·

Code: [mschwarzmueller/go-complete-guide-resources](#)